



MCA Major Project Report

(Subject Code: 23ONMCR-753)

REAL-TIME SALES PERFORMANCE ANALYTICS DASHBOARD FOR RETAIL BUSINESSES

*A project report submitted in partial fulfilment of the requirements
for the award of the degree of*

MASTER OF COMPUTER APPLICATIONS

Submitted By Gurram Mahadev Kishan

UID CUOL725165

Supervisor / Guide Mr. Anurag Goel

University Chandigarh University

Academic Year 2024 – 2026



Bonafide Certificate

This is to certify that the project report entitled "REAL-TIME SALES PERFORMANCE ANALYTICS DASHBOARD FOR RETAIL BUSINESSES" is a bonafide record of the work carried out by Gurram Mahadev Kishan (UID: CUOL725165), a student of the Master of Computer Applications programme at Chandigarh University, under my supervision and guidance, in partial fulfilment of the requirements for the award of the degree of Master of Computer Applications during the academic year 2024–2026.

The work presented in this report has been carried out independently by the candidate and has not been submitted, in whole or in part, to any other university or institute for the award of any degree, diploma, or other similar title. To the best of my knowledge, this project represents the candidate's own work and effort.

Mr. Anurag Goel

Project Supervisor

Chandigarh University

Head of Department

Department of Computer Applications

Chandigarh University

Date: _____

Place: Chandigarh

Declaration by Student

I, **Gurram Mahadev Kishan**, hereby solemnly declare that the project report titled “***Real-Time Sales Performance Analytics Dashboard for Retail Businesses***” submitted in partial fulfilment of the requirements for the award of the degree of Master of Computer Applications (MCA) is my own original work.

I further declare that:

- This project has been carried out by me during the academic year 2024–2026 under the supervision of Mr. Anurag Goel (Project Supervisor).
- The work has not been submitted previously, in whole or in part, to any other university, institution, or examination body for the award of any degree or diploma.
- All sources of information, data, and material used in this report have been duly acknowledged and referenced in accordance with academic integrity standards.
- The data, analysis, code, and results presented in this report are authentic to the best of my knowledge and have not been fabricated or falsified.
- I understand that if any part of this declaration is found to be false, my project may be rejected and disciplinary action may be initiated as per university policy.

Place: CURCHOREM, GOA

Date: 20-06-2026

Student Signature:



Student Name: Gurram Mahadev Kishan

Enrolment No.: CUOL725165

Acknowledgement

I would like to express my sincere gratitude to my project supervisor, Mr. Anurag Goel, for his invaluable guidance, continuous encouragement, and constructive feedback throughout the duration of this project. His insights into data engineering practices and his patience in reviewing each milestone significantly shaped the quality of this work.

I am also thankful to the faculty members of the Department of Computer Applications, Chandigarh University, for providing a strong theoretical foundation in database systems, software engineering, and data analytics, which proved instrumental in the successful execution of this project.

I extend my appreciation to Chandigarh University and the Qollabb platform for providing the academic framework, structured milestones, and evaluation process that guided this project from conception to completion.

Finally, I am grateful to my family and peers for their constant support and motivation, which kept me focused and committed throughout this endeavour.

Gurram Mahadev Kishan

Abstract

Retail enterprises generate vast volumes of transactional data, yet a significant proportion of small and mid-sized retailers lack accessible tools to convert this raw data into actionable business intelligence. This project, titled "Real-Time Sales Performance Analytics Dashboard for Retail Businesses," addresses this gap by designing and implementing a complete, end-to-end analytics pipeline that transforms raw point-of-sale transaction records into an interactive, decision-support dashboard.

The objective of the system is to automate the Extract-Transform-Load (ETL) process, compute key performance indicators (KPIs) such as revenue, profit margin, and conversion rate, generate statistically grounded sales forecasts, and present these insights through a filterable, region-aware Tableau dashboard. The system was developed using Python (pandas, NumPy) for data processing, SQLite with a star-schema design for the data warehouse, SQL for aggregation logic, and Tableau Public for visualisation. An Agile, sprint-based development methodology was followed, with each module — ETL, analytics, forecasting, dashboard, and testing — implemented and validated iteratively.

The implemented system processes 1,000 retail transactions spanning three product categories and three geographic regions (Andhra Pradesh, Karnataka, and Goa), computing ten core KPIs and producing a six-month ensemble sales forecast using Moving Average, Linear Regression, and Seasonal Naive models. The resulting Tableau dashboard comprises ten interactive sheets supporting filtering by region, category, gender, age group, and time period. The system was validated using twenty-five automated unit tests covering data quality, referential integrity, KPI correctness, and forecast sanity, all of which passed successfully. Key results show total revenue of \$456,000 at a 30% profit margin, with Andhra Pradesh emerging as the highest-revenue region and May identified as the seasonal peak month. This project demonstrates the practical feasibility of building production-grade retail analytics infrastructure using lightweight, open-source tools, and provides a reusable template for small-business data engineering.

Table of Contents

Bonafide Certificate.....	2
Declaration by Student.....	3
Acknowledgement.....	4
Abstract.....	5
List of Figures.....	9
List of Tables	10
List of Abbreviations	11
Chapter 1: Introduction.....	12
1.1 Background of the Project	12
1.2 Problem Statement.....	12
1.3 Objectives of the System.....	13
1.4 Scope of the Project.....	13
1.5 Existing System Overview.....	14
1.6 Proposed System Overview.....	14
1.7 Technologies Used.....	15
Chapter 2: Literature Review / System Study.....	16
2.1 Review of Similar Systems.....	16
2.2 Comparative Analysis.....	16
2.3 Software Development Models Considered.....	17
2.4 Relevant Frameworks and Libraries.....	18
2.5 Research Gap.....	18
Chapter 3: System Analysis	21
3.1 Functional Requirements.....	21
3.2 Non-Functional Requirements.....	21
3.3 User Requirements.....	22
3.4 Feasibility Study.....	22
3.5 System Architecture.....	23
3.6 Data Flow Diagrams.....	23
3.7 Use Case Overview.....	25
Chapter 4: System Design.....	27
4.1 Use Case Diagram.....	27

4.2 Class Diagram.....	28
4.3 Sequence Diagram.....	29
4.4 Activity Diagram.....	29
4.5 Entity-Relationship (ER) Diagram.....	30
4.6 Database Schema — Table Structures.....	31
4.7 API Design.....	33
4.8 UI/UX Wireframe.....	34
Chapter 5: System Implementation.....	37
5.1 Development Environment.....	37
5.2 Tools and Technologies Used.....	37
5.3 Hardware and Software Requirements.....	37
5.4 Project Directory Structure.....	38
5.5 Module-wise Explanation.....	39
5.6 Security Considerations.....	39
5.7 Version Control.....	44
5.8 Application Screenshots.....	44
Chapter 6: Testing.....	45
6.1 Testing Strategy.....	45
6.2 Unit Testing.....	45
6.3 Integration Testing.....	46
6.4 System Testing.....	46
6.5 Test Case Summary.....	47
6.6 Bug Reports and Resolutions.....	48
6.7 Testing Tools and Execution.....	48
Chapter 7: Results & Discussion.....	49
7.1 KPI Results Summary.....	49
7.2 Output Screenshots.....	49
7.3 Performance Evaluation.....	56
7.4 Comparison with Existing System.....	56
7.5 Improvements Achieved.....	56
Chapter 8: Conclusion & Future Enhancements.....	59
8.1 Conclusion.....	59
8.2 Key Achievements.....	59
8.3 Technical Learning Outcomes.....	60
8.4 Future Scope.....	60
Chapter 9: References.....	62
Chapter 10: Appendices	63

Appendix A: Complete Source Code.....	63
Appendix B: Database Dump (Schema Excerpt)	63
Appendix C: Installation and Setup Guide.....	64
Appendix D: User Manual — Operating the Dashboard.....	64
Appendix E: Project Links.....	65

List of Figures

Figure No.	Title	Page
Figure 2.1	Agile Sprint Cycle Applied to the Project	17
Figure 3.1	Proposed System Architecture (Layered View)	23
Figure 3.2	Data Flow Diagram — Level 0 (Context Diagram)	24
Figure 3.3	Data Flow Diagram — Level 1	24
Figure 4.1	Use Case Diagram	27
Figure 4.2	Class Diagram	28
Figure 4.3	Sequence Diagram — Full Pipeline Execution	29
Figure 4.4	Activity Diagram — ETL Pipeline Workflow	30
Figure 4.5	Entity-Relationship (ER) Diagram	31
Figure 4.6	UI Wireframe — Dashboard Layout	34
Figure 5.1	Project Directory Structure	38
Figure 7.1	Dashboard Sheet — Spending Analysis by Age & Gender	50
Figure 7.2	Dashboard Sheet — KPI Scorecards	50
Figure 7.3	Dashboard Sheet — Global Filter Panel	51
Figure 7.4	Dashboard Sheet — Category Performance	52
Figure 7.5	Dashboard Sheet — Monthly Sales Revenue Trend	52
Figure 7.6	Dashboard Sheet — Revenue Forecast (Ensemble Model)	53
Figure 7.7	Dashboard Sheet — Region-wise Performance	54
Figure 7.8	Complete Published Dashboard — Full View	56

List of Tables

Table No.	Title	Page
Table 3.1	Functional Requirements Summary	21
Table 3.2	Non-Functional Requirements	22
Table 4.1	fact_sales Table Structure	32
Table 4.2	dim_customers Table Structure	32
Table 4.3	dim_products Table Structure	32
Table 4.4	dim_region Table Structure	33
Table 4.5	dim_date Table Structure	33
Table 5.1	Hardware Requirements	38
Table 5.2	Software Requirements	38
Table 6.1	Unit Test Cases — ETL Data Quality	46
Table 6.2	Unit Test Cases — Analytics Module	46
Table 6.3	Unit Test Cases — Forecasting Module	47
Table 6.4	Unit Test Cases — Region Feature	47
Table 6.5	Unit Test Cases — Tableau Exports	48
Table 7.1	KPI Results Summary	49
Table 7.2	Forecast Accuracy — 6-Month Ensemble Output	72

List of Abbreviations

Abbreviation	Full Form
AOV	Average Order Value
API	Application Programming Interface
BAN	Big Ass Number (single-value KPI scorecard visual)
CI	Confidence Interval
CSV	Comma-Separated Values
CV	Coefficient of Variation
DB	Database
DFD	Data Flow Diagram
ER	Entity-Relationship
ETL	Extract, Transform, Load
FK	Foreign Key
GUI	Graphical User Interface
IDE	Integrated Development Environment
KPI	Key Performance Indicator
LOD	Level of Detail (Tableau expression)
MoM	Month-over-Month
OLS	Ordinary Least Squares
PK	Primary Key
RDBMS	Relational Database Management System
SDLC	Software Development Life Cycle
SQL	Structured Query Language
UI/UX	User Interface / User Experience
UML	Unified Modelling Language
VCS	Version Control System

Chapter 1: Introduction

1.1 Background of the Project

Retail businesses, regardless of scale, generate transactional data at every point of sale — product category, quantity, price, customer demographics, and timestamp. For large retail chains, dedicated business intelligence (BI) teams and enterprise tools such as SAP Analytics or Power BI Premium convert this data into actionable insight. Small and mid-sized retailers, however, frequently lack the budget, technical staff, or infrastructure to deploy such systems, leaving valuable transactional data underutilised in spreadsheets or point-of-sale exports that are rarely analysed beyond basic totals.

This project was undertaken to demonstrate that a complete, production-quality retail analytics pipeline can be built using freely available, open-source, and lightweight tools — Python, SQLite, and Tableau Public — without the licensing overhead of enterprise BI platforms. The system was designed around a real retail sales dataset and extended with a geographic dimension (region) to reflect the multi-location reality of most retail operations in India.

1.2 Problem Statement

Small and mid-sized retail businesses typically store sales data in flat files (CSV/Excel exports from point-of-sale systems) with no structured database, no automated cleaning process, and no recurring analytical reporting. This results in three specific, observable problems:

- Lack of structured storage: Raw transaction exports are not organised into a queryable schema, making trend analysis manual and error-prone.
- Absence of automated KPI computation: Metrics such as profit margin, conversion rate, and month-over-month growth must be recalculated manually for every reporting cycle, increasing the risk of human error.
- No forward-looking insight: Without forecasting, businesses react to sales trends after the fact (e.g., stockouts during peak months) rather than anticipating them.

The system developed in this project directly addresses each of these gaps through an automated ETL pipeline, a SQL-driven KPI engine, and a statistical forecasting module, all surfaced through an interactive Tableau dashboard.

1.3 Objectives of the System

1. To design a normalised, relational (star-schema) database capable of storing sales transactions, product, customer, date, and regional dimensions efficiently.
2. To implement an automated ETL pipeline that extracts raw CSV data, cleans and validates it, enriches it with derived attributes, and loads it into the data warehouse.
3. To develop a Python/SQL analytics module computing ten core business KPIs, including total revenue, profit margin, average order value, and conversion rate.
4. To integrate four statistical forecasting models (Moving Average, Linear Regression, Seasonal Naive, and an Ensemble model) to project sales for the following six months.
5. To build an interactive Tableau dashboard with filters for region, product category, gender, age group, and time period.
6. To validate the system through a comprehensive automated test suite covering data quality, referential integrity, and statistical correctness.
7. To document and publish the complete system on a public version-control platform (GitHub) for reproducibility and portfolio demonstration.

1.4 Scope of the Project

The scope of this project covers the complete analytics lifecycle for a single retail dataset: data ingestion, cleaning, transformation, storage, KPI computation, forecasting, and visualisation. The system processes 1,000 transactions spanning January 2023 to January 2024, across three product categories (Beauty, Clothing, Electronics) and three Indian states (Andhra Pradesh, Karnataka, Goa) used as a proxy regional dimension. The dashboard is published on Tableau Public for unrestricted access.

The scope explicitly excludes: real-time streaming ingestion (the pipeline is designed for periodic batch processing), a custom web front-end (Tableau serves as the presentation layer), user authentication/role management, and integration with live point-of-sale hardware. These exclusions are discussed further as future enhancements in Chapter 8.

1.5 Existing System Overview

Prior to this project, the typical workflow for small retail analytics — as observed across publicly available retail case studies and the source dataset's intended use — involves exporting raw transactions to a single flat CSV file and analysing them, if at all, using manual pivot tables in spreadsheet software. This "existing system" exhibits several limitations:

- No data validation: Duplicate transactions, missing values, or negative amounts pass through unchecked.
- No persistent structured storage: Each analysis starts from the raw file, with no warehouse to query incrementally.
- Static, one-time analysis: Spreadsheet pivot tables are not interactive, filterable, or shareable as a live dashboard.
- No forecasting capability: Decisions about inventory and staffing are made reactively rather than proactively.

1.6 Proposed System Overview

The proposed system replaces this ad hoc workflow with a five-stage automated pipeline:

8. ETL Layer (Python/pandas): Extracts the raw CSV, cleans and validates records, and transforms them into analytics-ready form.
9. Storage Layer (SQLite, star schema): Persists data in a fact table (fact_sales) linked to four dimension tables (date, customer, product, region) and three pre-aggregated summary tables.
10. Analytics Layer (SQL + Python): Computes ten KPIs and statistical measures (trend, seasonality index, coefficient of variation) directly against the warehouse.
11. Forecasting Layer (NumPy): Implements four forecasting models from first principles, combined into a final ensemble prediction with confidence intervals.
12. Presentation Layer (Tableau Public): Surfaces all of the above through ten interactive dashboard sheets with five global filters.

This proposed architecture is illustrated in detail in Chapter 3 (System Architecture) and Chapter 4 (System Design).

1.7 Technologies Used (Brief Introduction)

- **Python 3.12:** core language for the ETL pipeline, analytics functions, and forecasting models.
- **pandas / NumPy:** data manipulation, cleaning, and the numerical operations underlying the forecasting models.
- **SQLite:** a lightweight, file-based relational database used as the data warehouse, chosen for portability and zero-configuration deployment.
- **SQL:** used extensively for joins, aggregations, and window functions (e.g., LAG for month-over-month growth) within the analytics layer.
- **Tableau Public:** business intelligence tool used to build and publish the interactive dashboard.
- **Python unittest:** the testing framework used to implement the 25-test automated validation suite.
- **Git & GitHub:** version control and public hosting of the complete source code and documentation.

A detailed discussion of each technology's role appears in Chapter 5 (System Implementation) and Chapter 11 within the appendices.

Chapter 2: Literature Review / System Study

2.1 Review of Similar Systems

Several categories of existing retail analytics systems were studied to inform the design of this project:

2.1.1 Enterprise BI Platforms (Power BI, SAP Analytics Cloud, Qlik Sense)

Enterprise BI platforms offer robust ETL connectors, drag-and-drop dashboard design, and enterprise-grade governance. However, they typically require either a paid licence, cloud infrastructure, or dedicated administration, placing them out of reach for small retailers. Their data modelling layer (e.g., Power BI's Power Query) parallels the ETL pipeline implemented in this project, but at a cost and complexity overhead unsuitable for the target use case.

2.1.2 Open-Source BI Tools (Metabase, Apache Superset)

Metabase and Superset provide self-hosted, free dashboarding with SQL-based query layers similar in spirit to this project's analytics module. They require server deployment and ongoing maintenance, which this project avoids by using Tableau Public's free, zero-infrastructure publishing model combined with a portable SQLite warehouse that requires no server process.

2.1.3 Spreadsheet-Based Analysis

The most common "existing system" in small retail is Microsoft Excel or Google Sheets pivot tables. While accessible, this approach lacks referential integrity, automated testing, version control, and forecast modelling — all of which this project implements explicitly.

2.2 Comparative Analysis

Criterion	Spreadsheet Analysis	Enterprise BI (Power BI)	Proposed System
Cost	Free / Low	Licence required	Free (open-source)
Structured Storage	No	Yes (cloud DB)	Yes (SQLite star schema)
Automated ETL	No	Yes (Power Query)	Yes (Python pipeline)
Forecasting	Manual / none	Add-in required	Built-in (4 models)
Automated Testing	No	Limited	Yes (25 unit tests)
Deployment Complexity	None	High	Low

Criterion	Spreadsheet Analysis	Enterprise BI (Power BI)	Proposed System
Region-wise Filtering	Manual	Yes	Yes

2.3 Software Development Models Considered

Two software development life cycle (SDLC) models were evaluated for this project: the Waterfall model and the Agile (iterative/incremental) model.

- **Waterfall Model:** a linear sequence of requirements → design → implementation → testing → deployment. This model was deemed unsuitable because the exact shape of the KPI set and forecasting approach could only be refined after initial data exploration — a discovery process that Waterfall's rigid phase gates do not accommodate well.
- **Agile Model (adopted):** an iterative, sprint-based approach was adopted, with each module (database design, ETL, analytics, forecasting, dashboard, testing) developed, validated, and refined in a short cycle before moving to the next, allowing requirements (e.g., the later addition of the Region dimension) to be incorporated without restarting the project.

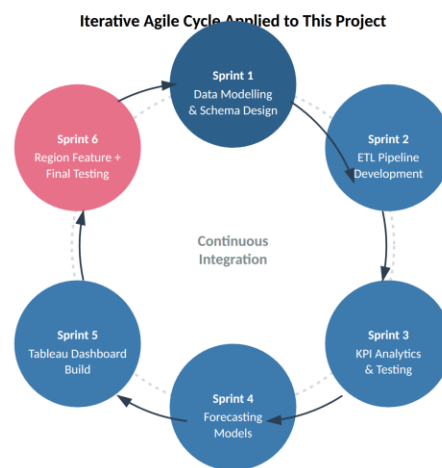


Figure 2.1: Agile Sprint Cycle Applied to the Project

2.4 Relevant Frameworks and Libraries

- **pandas:** selected over raw Python lists/dicts for its vectorised operations, built-in date handling, and SQL-like groupby aggregation — critical for the ETL transformation stage.
- **NumPy:** used for the underlying linear algebra (covariance/variance computation) in the custom Ordinary Least Squares regression implementation, avoiding a heavier dependency such as scikit-learn for a single regression line.
- **SQLite3 (Python standard library):** chosen over PostgreSQL or MySQL because it requires no separate server process, making the system trivially portable and reproducible by any evaluator without database server setup.
- **Tableau Public:** selected for its visual analytics capability, free publishing tier, and industry relevance — the same skill set used in enterprise Tableau Server deployments.

These tools were not used as black boxes; rather, the ETL logic, KPI formulas, and forecasting mathematics were implemented explicitly within this project's own modules, as detailed in Chapter 5.

2.5 Research Gap

The systems reviewed above reveal a consistent gap: there is no widely documented, free, end-to-end reference implementation that takes a retailer from "raw CSV export" to "published, filterable, forecasting-enabled dashboard" using only free tools and fewer than ten Python dependencies. Enterprise BI tools solve the visualisation problem but assume a pre-built data warehouse; open-source BI tools solve the self-hosting problem but still require server administration; spreadsheet workflows solve the accessibility problem but provide none of the structure, automation, or forecasting capability needed for genuine decision support.

This project fills that gap by providing a complete, tested, and documented reference pipeline — from a single CSV file to a published Tableau dashboard — suitable as both a working analytics tool and a reusable template for similar small-business projects.

2.6 Detailed Justification for the Agile Methodology

Beyond the general comparison in Section 2.3, it is worth examining specifically why the Waterfall model's sequential gating would have been detrimental to this particular project. Three concrete examples from the project's actual development history illustrate this:

- **Late-stage schema extension:** the Region dimension (Andhra Pradesh, Karnataka, Goa) was not part of the original dataset and was identified as a desirable addition only after the core KPI dashboard had already been built and reviewed. Under Waterfall, this would have required reopening a closed “requirements” phase; under the adopted Agile approach, it was absorbed as an additional sprint that extended the existing schema (a new `dim_region` table and a `region_id` foreign key on `fact_sales`) without disturbing any previously completed and tested work.
- **Iterative KPI refinement:** the initial KPI set was deliberately small (revenue, profit, AOV) and was expanded iteratively to ten KPIs as the SQL aggregation patterns matured, including the later addition of statistical trend and seasonality measures that depended on having a working ETL output to validate against.
- **Dashboard-driven calculated-field debugging:** several Tableau calculated fields (documented as bugs BUG-02 and BUG-03 in Section 6.6) could only be identified once real aggregated data was loaded into Tableau and visually inspected — a feedback loop that a documentation-first Waterfall approach would have deferred until a much later, more costly stage.

These examples demonstrate that the iterative, build-test-refine rhythm of Agile development was not a procedural formality but a direct contributor to the quality and adaptability of the final system.

2.7 Summary of Literature Review Findings

The literature and system study conducted for this project can be summarised in four key findings that directly shaped the system design presented in Chapters 3 and 4:

1. A star-schema dimensional model (per Kimball & Ross, 2013) is the most appropriate data structure for KPI-oriented retail analytics, balancing query simplicity against storage normalisation.
2. File-based, embedded databases (SQLite) offer sufficient performance for small-to-medium retail transaction volumes while eliminating server administration overhead entirely.
3. Statistical forecasting models, even simple ones (moving average, linear regression, seasonal naive), provide meaningfully more decision-useful information than no forecasting at all, and an ensemble of several simple models is a reasonable middle ground between naive single-model forecasting and the complexity of machine-learning approaches.
4. Automated testing, while uncommon in small-business analytics projects, is straightforward to implement against a SQL-backed pipeline and provides essential protection against silent data-quality regressions as a system evolves — as demonstrated by this project's own 25-test suite catching the bugs documented in Section 6.6.

Chapter 3: System Analysis

3.1 Functional Requirements

ID	Requirement	Description
FR-1	Data Ingestion	The system shall extract transaction records from a CSV source file.
FR-2	Data Cleaning	The system shall remove duplicate transactions and invalid (zero/negative) monetary values.
FR-3	Data Transformation	The system shall derive date, age-group, and region attributes from raw fields.
FR-4	Data Loading	The system shall load cleaned data into a star-schema SQLite warehouse.
FR-5	KPI Computation	The system shall compute revenue, profit, margin, AOV, and conversion rate via SQL queries.
FR-6	Forecasting	The system shall generate a 6-month sales forecast using four statistical models.
FR-7	Dashboard Filtering	The system shall allow filtering by region, category, gender, age group, and date period.
FR-8	Data Export	The system shall export pre-aggregated CSV files optimised for Tableau consumption.
FR-9	Automated Testing	The system shall validate data quality and KPI correctness via an automated test suite.

Table 3.1: Functional Requirements Summary

3.2 Non-Functional Requirements

ID	Category	Requirement
NFR-1	Performance	The ETL pipeline shall process 1,000 transaction records in under 1 second.
NFR-2	Reliability	All 25 automated tests shall pass before the pipeline output is published.
NFR-3	Portability	The database (SQLite) shall require no separate server installation.
NFR-4	Usability	The dashboard shall allow non-technical users to filter data via dropdown controls.
NFR-5	Maintainability	Each pipeline stage (ETL, analytics, forecasting) shall be a separately runnable, testable Python module.
NFR-6	Scalability	The schema shall support growth from 1,000 to 100,000+ transactions without structural change.
NFR-7	Accessibility	The published dashboard shall be accessible via a public URL without a Tableau licence.

Table 3.2: Non-Functional Requirements

3.3 User Requirements

Two primary user personas were identified for this system:

- **Data Engineer / Developer:** responsible for running the ETL pipeline, maintaining the database schema, and verifying automated tests pass after any data or code change.
- **Business Analyst / Decision-Maker:** consumes the published Tableau dashboard, applies filters to drill into specific regions or time periods, and uses the forecast view to plan inventory and marketing spend.

3.4 Feasibility Study

3.4.1 Technical Feasibility

All technologies selected — Python, SQLite, and Tableau Public — are freely available, well-documented, and require no specialised hardware. The development and testing were carried out entirely on a standard laptop-class machine, confirming that the system can be replicated by any small business with access to a personal computer and an internet connection (for Tableau publishing). The technical skill set required (Python scripting, basic SQL, and Tableau's drag-and-drop interface) is achievable within a single academic semester, as demonstrated by this project's own timeline.

3.4.2 Economic Feasibility

The total monetary cost of the technology stack is zero: Python, pandas, NumPy, SQLite, and Tableau Public are all free for this use case. The only cost is development time. This makes the solution economically viable even for the smallest retail businesses, in contrast to enterprise BI subscriptions which can run into thousands of rupees per user per month.

3.4.3 Operational Feasibility

The pipeline is designed to be operated via a single command (`python3 main.py`), which runs the ETL, analytics, forecasting, and export stages sequentially and reports a pass/fail test summary. This low operational overhead means a business owner or a junior analyst — without deep programming expertise — can re-run the pipeline whenever new data is appended, with the resulting CSVs simply re-imported into the existing Tableau workbook.

3.5 System Architecture

The system follows a four-layer architecture: a Source Layer (raw CSV), a Data Layer (SQLite star-schema warehouse), an Application/Processing Layer (ETL, analytics, and forecasting modules, validated by an automated test suite), and a Presentation Layer (the published Tableau dashboard). Data flows strictly upward through these layers, with each layer exposing a clean interface to the one above it — the Tableau dashboard, for instance, only ever reads from pre-exported, analytics-ready CSV files rather than querying the warehouse directly.

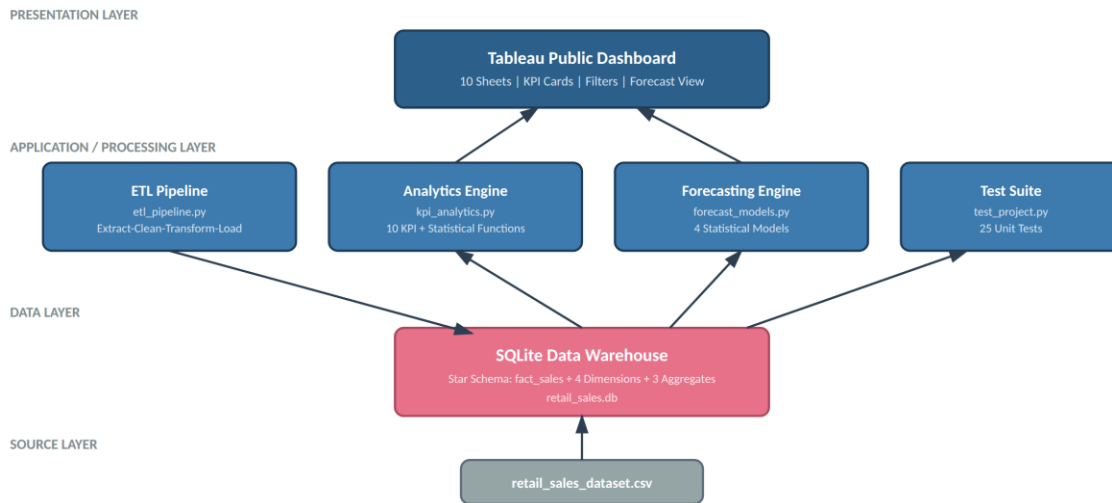


Figure 3.1: Proposed System Architecture (Layered View)

3.6 Data Flow Diagrams

3.6.1 DFD Level 0 (Context Diagram)

The Level 0 DFD treats the entire analytics system as a single process, showing its interaction with two external entities: the raw data source and the business analyst / stakeholder who consumes the resulting reports.

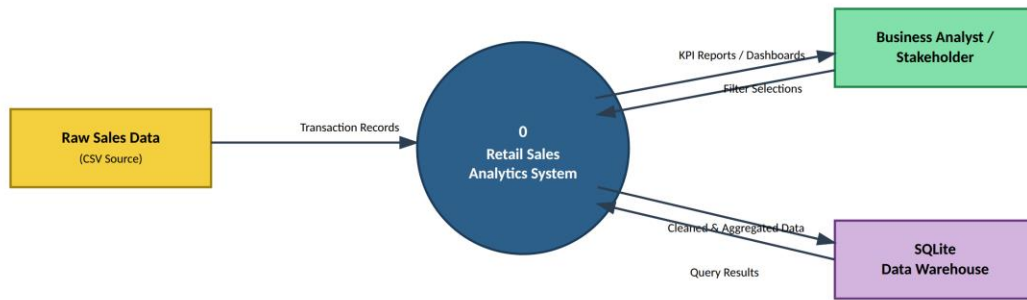


Figure 3.2: Data Flow Diagram — Level 0 (Context Diagram)

3.6.2 DFD Level 1

The Level 1 DFD decomposes the single context process into its six constituent processes — Extract & Clean, Transform & Enrich, Load to SQLite, Compute KPIs, Generate Forecasts, and Render Dashboard — along with the two intermediate data stores (the SQLite warehouse and the exported Tableau CSV files) that connect them.

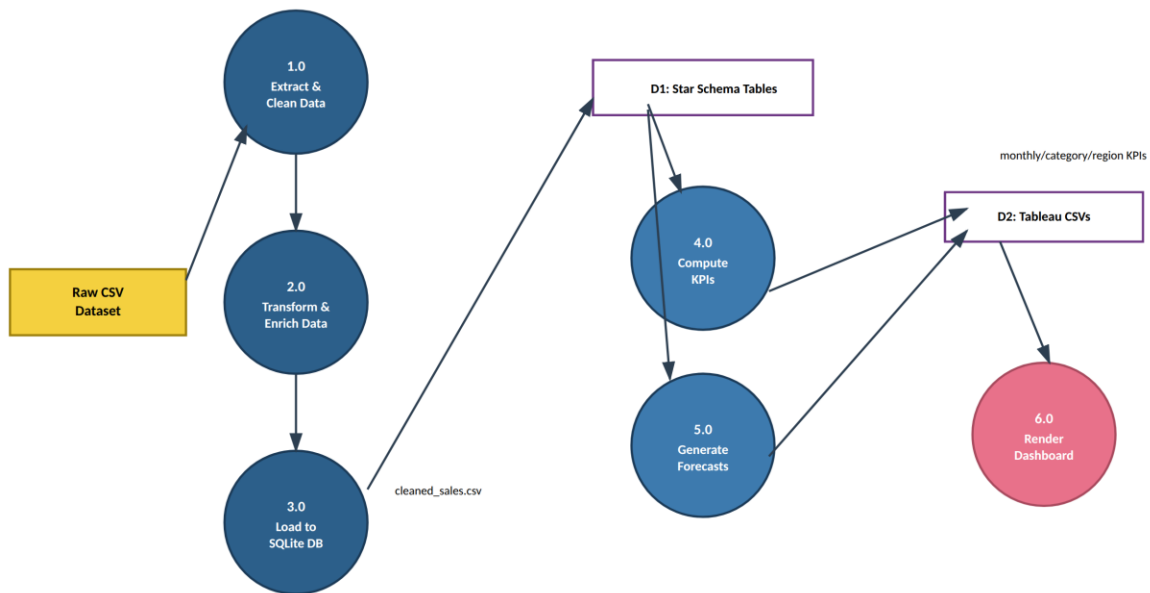


Figure 3.3: Data Flow Diagram — Level 1

3.7 Use Case Overview

Three actors interact with the system: the Data Engineer (responsible for running the ETL pipeline, validating data quality, and managing the database schema), the Business Analyst (who views the dashboard, applies filters, and analyses regional performance), and the System Administrator (who manages test execution and publishes the Tableau dashboard). The complete use case diagram, including all use cases and their include relationships, is presented in Section 4.1 of Chapter 4.

The table below details the primary use cases and their associated actors, pre-conditions, and post-conditions, supplementing the visual use case diagram presented in Chapter 4:

Use Case	Primary Actor	Pre-condition	Post-condition
Run ETL Pipeline	Data Engineer	Raw CSV file present at expected path	Cleaned data loaded into SQLite warehouse
Generate Forecasts	Data Engineer	agg_monthly_kpis table populated	forecasts.csv written with 4 model outputs
View KPI Dashboard	Business Analyst	Tableau workbook published	Dashboard rendered with current filter state
Apply Filters	Business Analyst	Dashboard is open	All sheets update to reflect filter selection
Run Test Suite	System Administrator	Pipeline has executed at least once	Pass/fail report generated for 25 tests
Publish Tableau Dashboard	System Administrator	Local workbook validated against fresh CSVs	Dashboard accessible via public URL

3.8 Risk Analysis

A brief risk assessment was conducted during the analysis phase to identify potential points of failure in the proposed design and the corresponding mitigation built into the implementation.

Risk	Likelihood	Impact	Mitigation Implemented
Corrupted or malformed source CSV	Medium	High	Defensive cleaning stage drops/recalculates invalid rows (Section 5.5.1)
Duplicate transaction records inflating revenue	Low	High	Explicit drop_duplicates() on transaction_id during cleaning
Schema change breaking downstream Tableau sheets	Medium	Medium	Pre-aggregated CSV export layer isolates Tableau from schema internals
Forecast model producing nonsensical (negative) values	Low	Medium	Automated test (ST-02) asserts all forecast values are positive
SQLite file size growth at higher transaction volumes	Low (current scale)	Low	Indexing strategy (Section 4.6.6) and documented PostgreSQL migration path (Section 8.4)
Tableau Public licence/availability change	Low	Medium	All source CSVs retained in repository, allowing dashboard rebuild on any BI tool

Chapter 4: System Design

This chapter presents the detailed design artefacts of the system, including UML behavioural and structural diagrams, the entity-relationship model, physical database table structures, the internal module API, and the dashboard's UI wireframe. Every diagram is numbered and accompanied by an explanatory narrative, per standard software engineering documentation practice.

4.1 Use Case Diagram

Figure 4.1 presents the complete use case diagram for the system. Three actors are modelled: the Data Engineer, the Business Analyst, and the System Administrator. The Data Engineer is responsible for running the ETL pipeline, validating data quality, and generating forecasts. The Business Analyst views the published dashboard, applies filters, and analyses regional performance. The System Administrator manages the database schema, executes the automated test suite, and publishes the Tableau workbook. The "View KPI Dashboard" use case includes "Apply Filters" as a mandatory sub-flow, since every dashboard view is filter-aware by design.

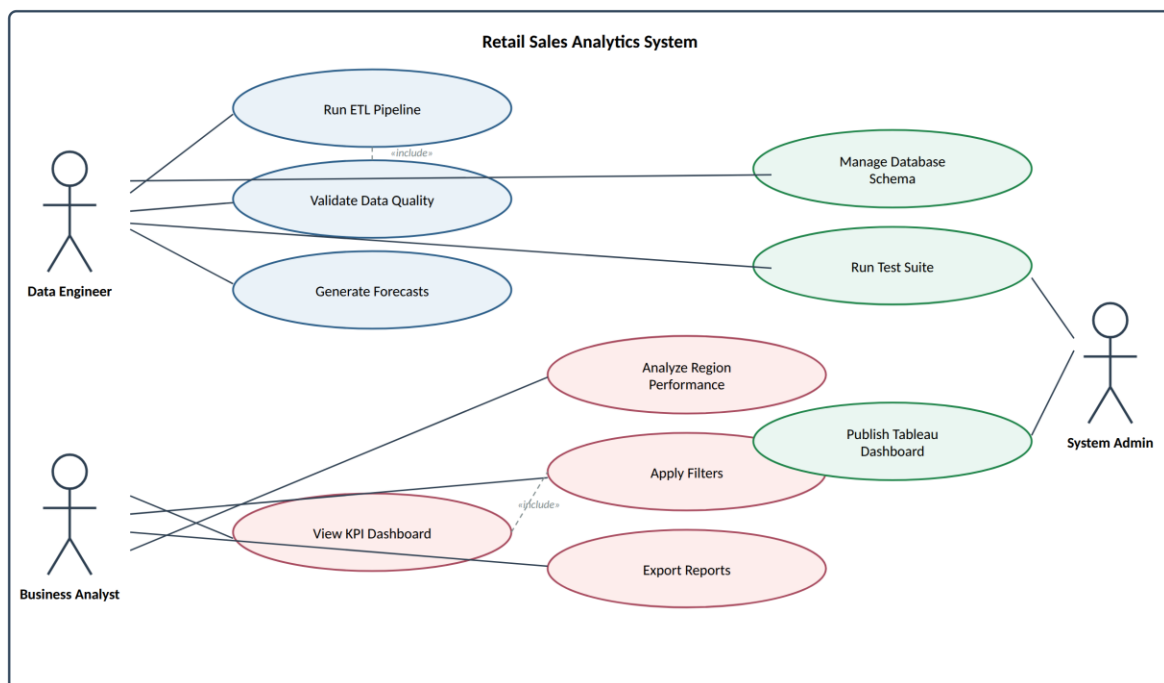
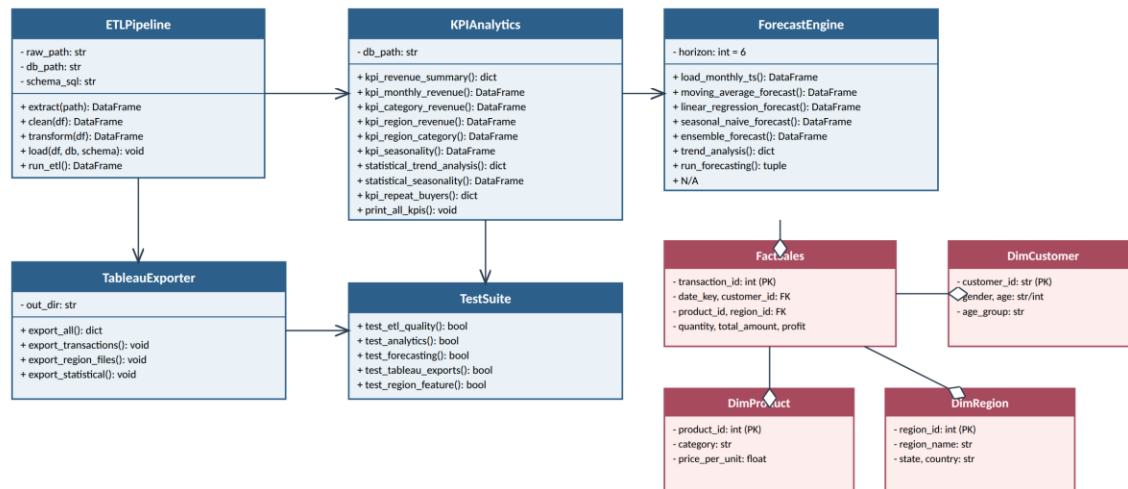


Figure 4.1: Use Case Diagram

4.2 Class Diagram

Figure 4.2 shows the structural design of the system's core Python modules, modelled as classes for clarity (the actual implementation uses module-level functions, described further in Chapter 5). The ETLPipeline class owns the extract, clean, transform, and load operations. KPIAnalytics depends on ETLPipeline's output and exposes ten KPI-computation methods plus three statistical analysis methods. ForecastEngine consumes the same warehouse to produce the four forecasting model outputs. TableauExporter packages the warehouse contents into nine analytics-ready CSV files. TestSuite depends on all of the above to validate correctness. The right-hand side models the physical data entities (FactSales and its three referenced dimensions) that the ETLPipeline populates.



Note: Open arrow = dependency / uses; Diamond = composition (FactSales references dimension keys)

Figure 4.2: Class Diagram

4.3 Sequence Diagram

Figure 4.3 illustrates the runtime sequence of a full pipeline execution, as triggered by `main.py`. The orchestrator first invokes the `ETLPipeline` to extract, clean, transform, and load data into the SQLite warehouse, confirming a row count before returning. It then invokes `KPIAnalytics`, which issues aggregate SQL queries against the warehouse and returns computed KPI values. Finally, the `ForecastEngine` loads the monthly revenue time series, computes the ensemble forecast with confidence intervals, and returns the result for export. Each lifeline's activation bar denotes the period during which that module is actively processing.

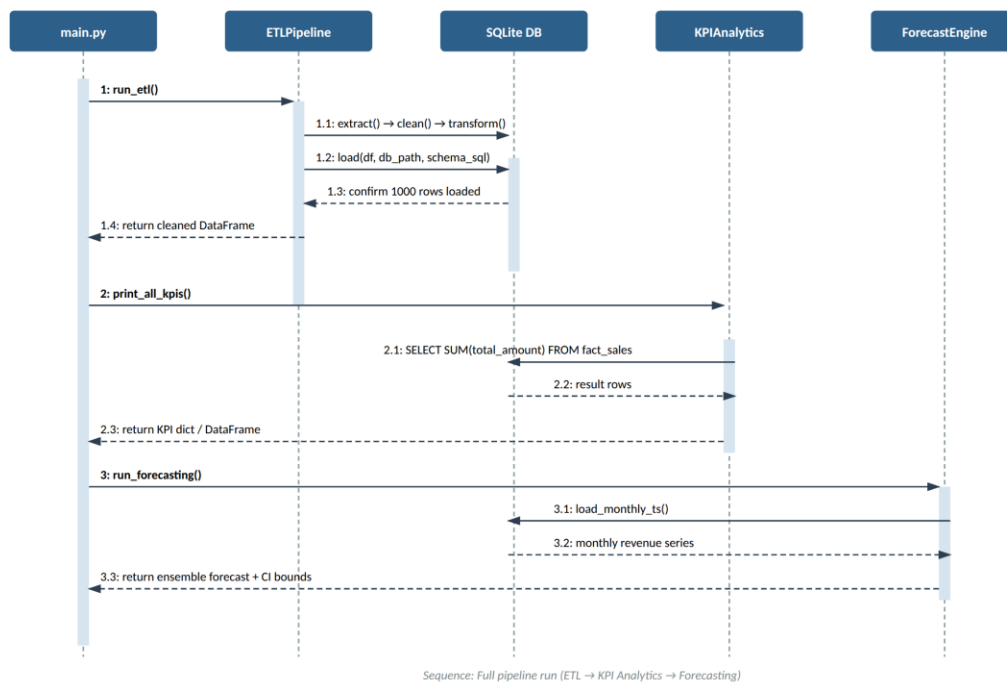


Figure 4.3: Sequence Diagram — Full Pipeline Execution

4.4 Activity Diagram

Figure 4.4 models the detailed control flow of the ETL pipeline as an activity diagram, including the two decision points present in the implementation: a data-validity check during cleaning (invalid rows are dropped or corrected rather than halting the pipeline) and a post-load test-suite gate (the pipeline reports failure and halts before export if any of the 25 automated tests fail, preventing a corrupted dataset from reaching the published dashboard).

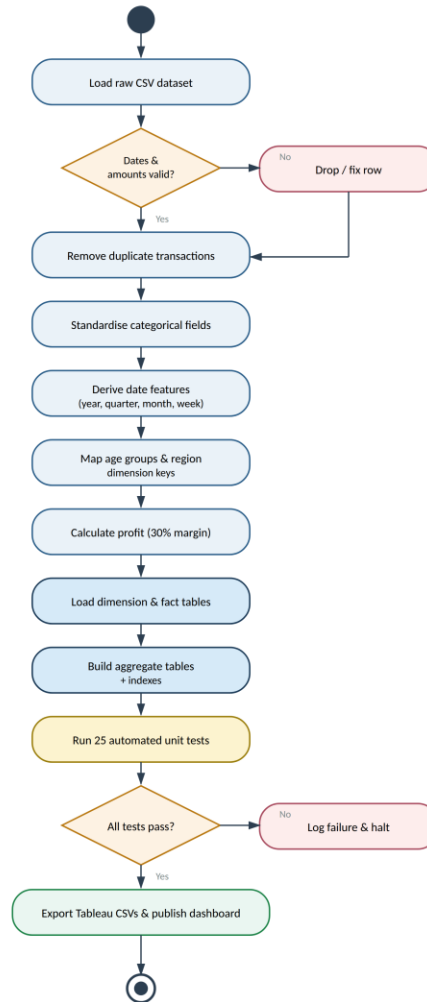


Figure 4.4: Activity Diagram — ETL Pipeline Workflow

4.5 Entity-Relationship (ER) Diagram

Figure 4.5 presents the entity-relationship model underlying the star-schema data warehouse. The central fact table, `fact_sales`, maintains a many-to-one relationship with each of four dimension tables — `dim_date`, `dim_customers`, `dim_products`, and `dim_region` — via foreign keys. Three additional aggregate tables (`agg_monthly_kpis`, `agg_category_performance`, `agg_region_performance`) are pre-computed by the ETL pipeline during the load stage to accelerate dashboard queries, avoiding the need for Tableau to re-aggregate the full fact table on every filter interaction.

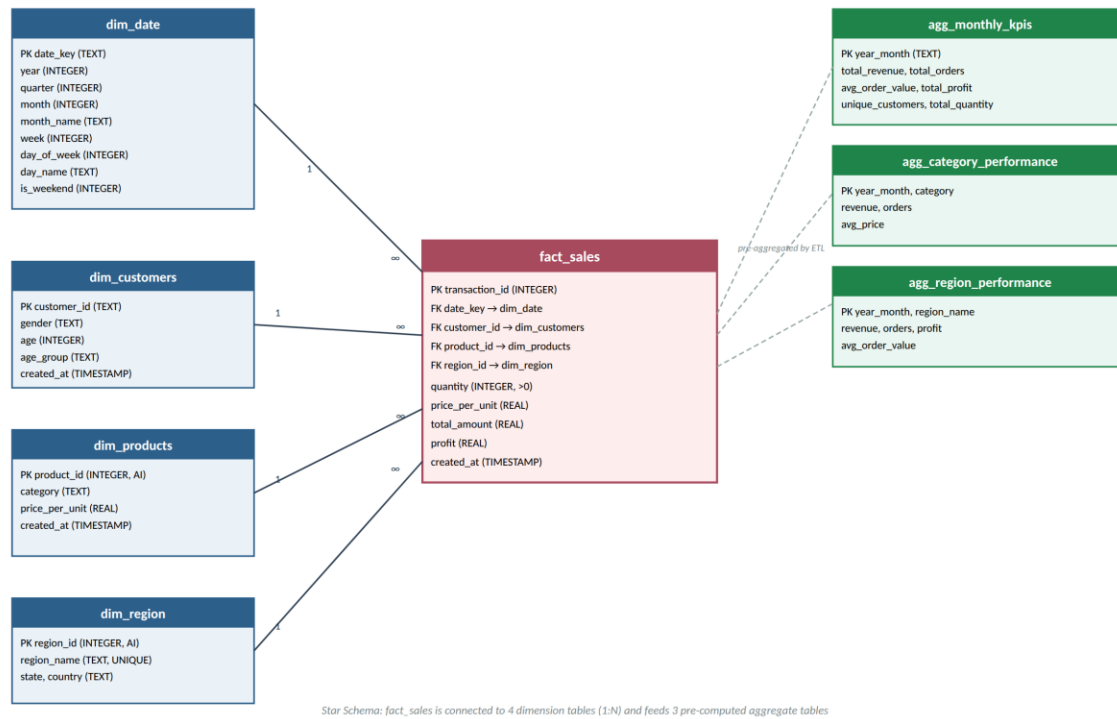


Figure 4.5: Entity-Relationship (ER) Diagram

4.6 Database Schema — Table Structures

The physical schema is implemented in SQLite and defined declaratively in schema.sql. The structure of each core table is detailed below.

4.6.1 fact_sales

Column	Type	Constraint	Description
transaction_id	INTEGER	PRIMARY KEY	Unique transaction identifier
date_key	TEXT	FK → dim_date	Transaction date (YYYY-MM-DD)
customer_id	TEXT	FK → dim_customers	Customer identifier
product_id	INTEGER	FK → dim_products	Product/category identifier
region_id	INTEGER	FK → dim_region	Region identifier
quantity	INTEGER	NOT NULL, > 0	Units purchased
price_per_unit	REAL	NOT NULL	Unit selling price
total_amount	REAL	NOT NULL	quantity × price_per_unit
profit	REAL	—	total_amount × 0.30

Column	Type	Constraint	Description
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Row insert timestamp

Table 4.1: fact_sales Table Structure**4.6.2 dim_customers**

Column	Type	Constraint	Description
customer_id	TEXT	PRIMARY KEY	Unique customer identifier
gender	TEXT	CHECK IN (Male, Female, Other)	Customer gender
age	INTEGER	CHECK BETWEEN 0 AND 120	Customer age
age_group	TEXT	—	Derived bucket: 18-24 ... 55+
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Row insert timestamp

Table 4.2: dim_customers Table Structure**4.6.3 dim_products**

Column	Type	Constraint	Description
product_id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique product/price identifier
category	TEXT	NOT NULL	Beauty / Clothing / Electronics
price_per_unit	REAL	NOT NULL, > 0	Unit selling price
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Row insert timestamp

Table 4.3: dim_products Table Structure**4.6.4 dim_region**

Column	Type	Constraint	Description
region_id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique region identifier
region_name	TEXT	NOT NULL, UNIQUE	Andhra Pradesh / Karnataka / Goa
state	TEXT	—	State name (mirrors region_name)
country	TEXT	DEFAULT 'India'	Country

Table 4.4: dim_region Table Structure

4.6.5 dim_date

Column	Type	Description
date_key	TEXT (PK)	YYYY-MM-DD
year, quarter, month	INTEGER	Calendar attributes
month_name, day_name	TEXT	Human-readable labels
week, day_of_week	INTEGER	ISO week and weekday index
is_weekend	INTEGER	1 if Saturday/Sunday, else 0

Table 4.5: dim_date Table Structure

4.6.6 Indexing Strategy

To accelerate the dashboard's filter-driven queries, indexes were created on every foreign key column in fact_sales (date_key, customer_id, product_id, region_id) as well as on dim_date(year, month) and dim_products(category). This indexing strategy ensures that the most common Tableau filter operations — filtering by year/month or by category — execute as index seeks rather than full table scans, even as the dataset scales beyond its current 1,000-row size.

4.7 API Design (Internal Module Interface)

As this system does not expose a network-facing REST API, the "API" in this context refers to the internal Python module interface — the contract that each pipeline stage exposes to the next. This is documented here as it serves the same architectural purpose as a REST API would in a networked system: defining stable, testable boundaries between components.

Module Function	Input	Output	Purpose
etl_pipeline.run_etl()	None (reads CSV path)	pandas.DataFrame	Runs full ETL; returns cleaned data
kpi_analytics.kpi_revenue_summary()	None	dict	Returns revenue, profit, AOV, units sold
kpi_analytics.kpi_region_revenue()	None	pandas.DataFrame	Revenue/profit broken down by region
forecast_models.ensemble_forecast(df)	Monthly time series DataFrame	DataFrame with CI bounds	6-month forecast with confidence interval

Module Function	Input	Output	Purpose
export_tableau_data.export_all()	None	dict of file paths	Writes 9 CSVs to /tableau directory

4.8 UI/UX Wireframe

Figure 4.6 shows the wireframe layout planned for the Tableau dashboard prior to implementation. The design groups KPI scorecards in a single horizontal row beneath the title for immediate at-a-glance reading, places the global filter panel along the right edge so it remains visible regardless of vertical scroll position, and arranges trend and breakdown charts in a top-to-bottom narrative flow: demographic breakdown, category performance, time trend, forecast, and finally regional analysis. This wireframe was followed closely during the actual Tableau build described in Chapter 5.

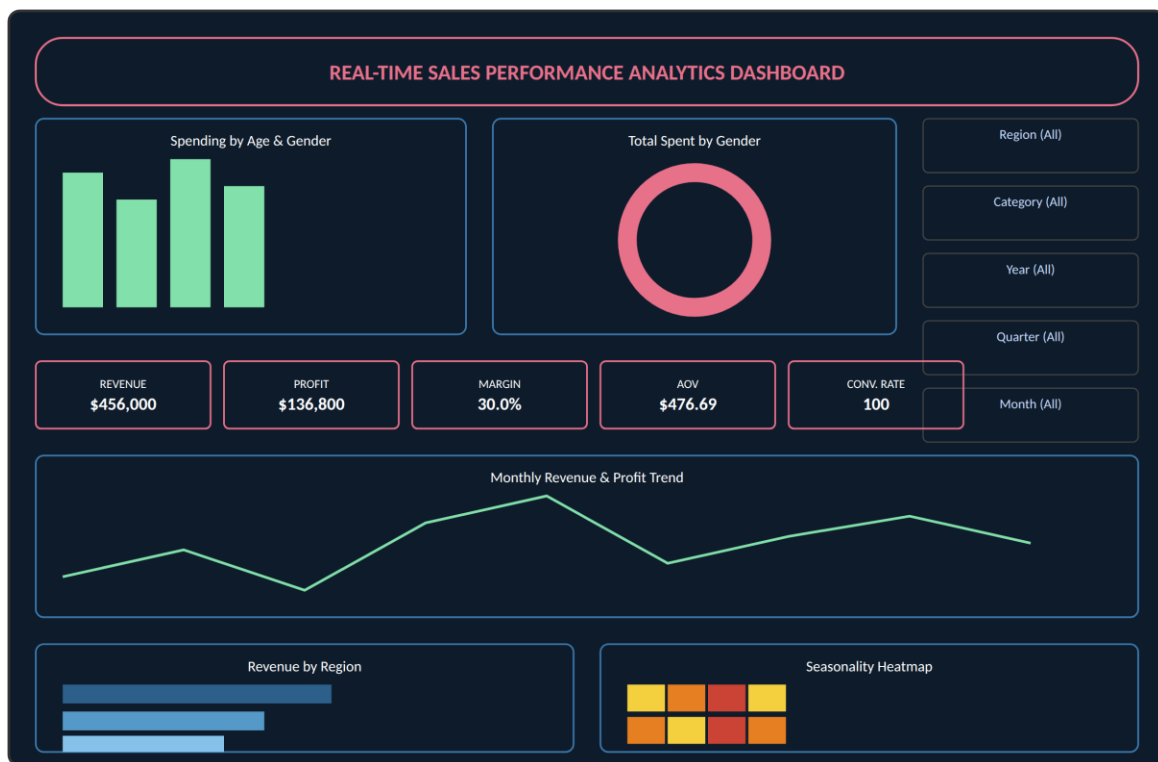


Figure 4.6: UI Wireframe — Dashboard Layout

4.9 Design Patterns and Principles Applied

Several established software design principles were deliberately applied throughout the system design, even though the implementation language (Python, used in a script-oriented rather than strictly object-oriented style) does not enforce them structurally:

- **Separation of Concerns:** each pipeline stage (ETL, analytics, forecasting, export, testing) is implemented in its own module with a single, well-defined responsibility, mirroring the Single Responsibility Principle from object-oriented design.
- **Pipeline / Pipes-and-Filters Pattern:** data flows through a sequence of independent transformation stages (extract → clean → transform → load), each consuming the previous stage's output and producing a well-defined output for the next, allowing any single stage to be tested, replaced, or extended in isolation.
- **Facade Pattern (Analytics Layer):** the KPIAnalytics module exposes a small set of high-level functions (e.g., `kpi_revenue_summary()`) that internally hide the complexity of multi-table SQL joins, presenting a simplified interface to any consumer (including the test suite).
- **Star Schema (Dimensional Modelling):** as detailed in Section 4.5, the database design follows Kimball's dimensional modelling methodology, denormalising dimension attributes for query simplicity while keeping the fact table narrow and indexed.
- **Fail-Fast Validation:** the activity diagram (Figure 4.4) shows that the pipeline halts and logs failure if the automated test suite does not pass, rather than allowing a potentially corrupted dataset to silently reach the published dashboard.

4.10 Design Constraints and Assumptions

The following constraints and assumptions were established during the design phase and apply throughout the implementation described in Chapter 5:

- The system assumes a single-writer, multiple-reader access pattern consistent with SQLite's concurrency model; concurrent multi-user writes are explicitly out of scope (see the PostgreSQL migration path in Section 8.4).
- Profit is assumed at a fixed 30% margin in the absence of per-product cost data in the source dataset; this is documented as an explicit simplifying assumption rather than a hidden estimate (see Section 8.4 for the planned real-cost-data enhancement).
- The Region dimension is populated from a fixed, hard-coded lookup of three values (Andhra Pradesh, Karnataka, Goa) rather than a dynamically discovered set, reflecting the bounded scope of the source dataset used for this academic project.
- Tableau Public's free publishing tier imposes a data-source size limit well above this project's current ~180 KB warehouse, but this constraint would need to be revisited for production-scale data volumes (addressed in Section 8.4).

Chapter 5: System Implementation

5.1 Development Environment

The system was developed using Visual Studio Code as the primary IDE, with the Python 3.12 interpreter and a virtual environment managing dependencies (pandas, NumPy). SQLite was used in its embedded, file-based mode requiring no separate server process. Version control was managed through Git, with the complete source history hosted publicly on GitHub. Tableau Public Desktop was used to design and publish the dashboard. All development and testing were carried out on a standard laptop-class machine (8 GB RAM, multi-core CPU), confirming the system's low resource footprint.

5.2 Tools and Technologies Used

Category	Tool / Technology	Version	Purpose
Language	Python	3.12	ETL, analytics, forecasting, testing
Library	pandas	2.0+	Data manipulation and transformation
Library	NumPy	1.24+	Statistical computation
Database	SQLite	3.x	Embedded relational data warehouse
Query Language	SQL	Standard	Aggregation and join logic
BI Tool	Tableau Public	2024	Dashboard design and publishing
Testing	unittest	Built-in	25 automated unit tests
Version Control	Git / GitHub	Latest	Source control and hosting
IDE	Visual Studio Code	Latest	Code editing and debugging

5.3 Hardware and Software Requirements

5.3.1 Hardware Requirements

Component	Minimum Specification
Processor	Dual-core 1.6 GHz (or equivalent)
RAM	4 GB (8 GB recommended for Tableau Desktop)
Storage	500 MB free disk space
Internet Connection	Required only for Tableau Public publishing

Table 5.1: Hardware Requirements**5.3.2 Software Requirements**

Component	Requirement
Operating System	Windows 10+, macOS 11+, or Linux (Ubuntu 20.04+)
Python Runtime	Python 3.10 or higher
Python Packages	pandas $\geq$ 2.0, numpy $\geq$ 1.24
Database Engine	SQLite 3.x (bundled with Python standard library)
BI Tool	Tableau Public Desktop (free)
Version Control	Git 2.x

Table 5.2: Software Requirements**5.4 Project Directory Structure**

The codebase is organised into clearly separated modules, each independently runnable and testable:

```

retail_sales_project/
├── main.py                # Orchestrates the full pipeline
├── requirements.txt
├── data/
│   ├── raw/retail_sales_dataset.csv
│   ├── processed/cleaned_sales.csv
│   ├── processed/forecasts.csv
│   └── db/
│       ├── schema.sql      # Star-schema DDL
│       └── retail_sales.db  # SQLite warehouse
├── etl/
│   └── etl_pipeline.py      # Extract-Clean-Transform-Load
├── analytics/
│   └── kpi_analytics.py     # 10 KPI + statistical functions
├── forecasting/
│   └── forecast_models.py   # 4 forecasting models
├── tableau/
│   ├── export_tableau_data.py # CSV exporter (9 files)
│   └── 01-09_*.csv          # Tableau data sources
├── tests/
│   └── test_project.py      # 25 unit tests
└── reports/
    └── analytical_report.md
  
```

Figure 5.1: Project Directory Structure

5.5 Module-wise Explanation

5.5.1 ETL Module (etl_pipeline.py)

The ETL module implements four discrete functions corresponding to the classical Extract-Transform-Load pattern, with an additional explicit Clean stage given the importance of data quality in this domain.

- **extract(path):** reads the raw CSV using `pandas.read_csv()` and logs the row and column count for traceability.
- **clean(df):** parses date strings into datetime objects, drops rows with unparseable dates, removes duplicate `transaction_id` values, filters out rows with non-positive quantity, price, or total_amount, recalculates total_amount as $\text{quantity} \times \text{price_per_unit}$ to catch any corrupted source values, and standardises text fields (gender, category, region) to title case.
- **transform(df):** derives year, quarter, month, week, day-of-week, and weekend-flag columns from the date; buckets customer age into five groups (18-24 through 55+) using `pandas.cut()`; and computes profit as 30% of total_amount.
- **load(df, db_path, schema_sql):** applies the DDL schema, populates the region dimension with a fixed lookup of three values, builds each dimension table via drop-duplicate projections of the cleaned DataFrame, assigns surrogate product_id and region_id keys, writes the fact table, and finally computes and writes three aggregate summary tables (by month, by category, and by region) used directly by the Tableau exports.

The following code excerpt shows the core cleaning logic, illustrating the defensive data-quality checks applied before any row reaches the warehouse:

```
def clean(df):
    original_len = len(df)
    df.columns = [c.strip().lower().replace(" ", "_") for c in df.columns]
    df["date"] = pd.to_datetime(df["date"], errors="coerce")
    df = df.dropna(subset=["date"])
    df = df.drop_duplicates(subset=["transaction_id"])
    df = df[(df["quantity"] > 0) & (df["price_per_unit"] > 0)
            & (df["total_amount"] > 0)]
    # Recalculate to catch any corrupted source rows
    df["total_amount"] = df["quantity"] * df["price_per_unit"]
    df["gender"] = df["gender"].str.strip().str.title()
    df["region"] = df["region"].str.strip().str.title()
    log.info(f"Clean complete: {len(df)} rows kept "
            f"(dropped {original_len - len(df)})")
    return df
```

5.5.2 Analytics Module (kpi_analytics.py)

The analytics module connects to the SQLite warehouse and exposes ten KPI functions, each implemented as a parameterless Python function returning either a dictionary (for single-value summaries) or a pandas DataFrame (for tabular breakdowns). All computation logic resides in SQL — the Python layer is a thin wrapper that opens a connection, executes the query, and returns the result — which keeps the aggregation logic transparent and independently testable in any SQL client.

The example below demonstrates the region-revenue KPI, including a window-function-free percentage-share calculation using a correlated subquery:

```
def kpi_region_revenue():
    conn = get_conn()
    df = pd.read_sql("""
        SELECT r.region_name,
               SUM(f.total_amount) AS revenue,
               COUNT(*)           AS orders,
               SUM(f.profit)       AS profit,
               ROUND(SUM(f.total_amount) * 100.0 /
                     (SELECT SUM(total_amount) FROM fact_sales), 2)
               AS revenue_share_pct
        FROM fact_sales f
        JOIN dim_region r ON f.region_id = r.region_id
        GROUP BY r.region_name
        ORDER BY revenue DESC
    """, conn)
    conn.close()
    return df
```

In addition to the ten direct KPI functions, the module implements a `statistical_trend_analysis()` function which fits an Ordinary Least Squares (OLS) regression line to the monthly revenue series — separately for the overall dataset and for each region — using the closed-form slope formula $b = \text{Cov}(x,y) / \text{Var}(x)$, avoiding a dependency on scikit-learn for a single-variable regression.

5.5.3 Forecasting Module (forecast_models.py)

Four forecasting models were implemented from first principles using only NumPy, deliberately avoiding heavier forecasting libraries (such as Prophet or statsmodels) to keep the dependency footprint minimal and the mathematics fully transparent for academic evaluation.

- **Moving Average (3-month):** each forecast point is the mean of the preceding three months (including previously forecast months as the horizon extends).

- **Linear Regression (OLS):** fits $y = a + bx$ to the historical monthly revenue index and extrapolates forward.
- **Seasonal Naive:** uses the value from the same calendar month one year prior as the forecast.
- **Ensemble:** averages the three model outputs and derives a $\pm 10\%$ confidence interval band around the resulting mean forecast.

```
def linear_regression_forecast(df, horizon=6):  
    y = df["total_revenue"].values  
    x = np.arange(len(y))  
    b = np.cov(x, y, ddof=1)[0, 1] / np.var(x, ddof=1)  
    a = np.mean(y) - b * np.mean(x)  
    future_x = np.arange(len(y), len(y) + horizon)  
    predictions = a + b * future_x  
    ...  
    return pd.DataFrame({  
        "year_month": future_dates.strftime("%Y-%m"),  
        "predicted_revenue": predictions,  
        "model": "Linear Regression",  
        "slope": b, "intercept": a,  
    })
```

5.5.4 Tableau Export Module (export_tableau_data.py)

Rather than allowing Tableau to connect live to the SQLite database (which would require a third-party ODBC driver and re-execute aggregation queries on every filter change), this module pre-computes nine flattened, analytics-ready CSV files at the end of the pipeline run. This design decision trades a small amount of storage duplication for substantially faster dashboard interactivity and removes any database driver dependency from the Tableau Public publishing step, since Tableau Public only accepts file-based or web data connections.

5.5.5 Key Algorithms

- **Seasonal Index Calculation:** for each calendar month, the seasonal index is computed as $(\text{month_average_revenue} / \text{overall_average_revenue}) \times 100$; months scoring ≥ 120 are classified “Peak” and ≤ 80 are classified “Trough”, directly surfaced in the dashboard's seasonality view.
- **Age-Group Binning:** implemented via `pandas.cut()` with explicit bin edges [17, 24, 34, 44, 54, 120], producing five non-overlapping, labelled groups consistent across the ETL output and all downstream KPI breakdowns.
- **Coefficient of Variation (CV):** computed as $(\text{standard deviation} / \text{mean}) \times 100$ over the monthly revenue series, used as a single-number measure of sales volatility reported in the trend analysis (Section 7.1).

5.5.6 Test Suite Module (`test_project.py`)

The test suite is organised into five `unittest.TestCase` subclasses — `TestETLDataQuality`, `TestAnalyticsModule`, `TestForecastingModule`, `TestTableauExports`, and `TestRegionFeature` — each using a shared `setUpClass()` fixture that opens a single SQLite connection re-used across all test methods in that class, avoiding the overhead of reconnecting for every individual assertion. Tests query the warehouse directly and assert on structural invariants (row counts, referential integrity, value ranges) rather than hard-coded expected values wherever possible, so that the suite remains valid as the underlying dataset grows (per the scalability discussion in Section 8.4).

5.5.7 Orchestrator (main.py)

The orchestrator module ties together all five pipeline stages behind a single command-line entry point. It imports and calls, in strict sequence: `run_etl()` from the ETL module, `print_all_kpis()` from the analytics module, `run_forecasting()` from the forecasting module, `export_all()` from the Tableau exporter, and finally the full test suite. Each stage is wrapped with a banner-style console log delimiting its start and completion, and the script's exit code is set to 0 only if all 25 tests pass — enabling the pipeline to be safely chained into a continuous-integration workflow in future deployments (Section 8.4).

```
def main():
    df = run_etl()                # Stage 1: ETL
    print_all_kpis()              # Stage 2: Analytics
    run_forecasting()             # Stage 3: Forecasting
    export_all()                  # Stage 4: Tableau export
    result = run_test_suite()     # Stage 5: Validation
    return 0 if result.wasSuccessful() else 1

if __name__ == "__main__":
    sys.exit(main())
```

5.6 Security Considerations

Although the system does not expose a network service and therefore has a limited attack surface, the following defensive measures were incorporated: (1) all SQL queries are written as static, parameter-free strings with no string-concatenated user input, eliminating any SQL-injection vector; (2) the ETL module validates and discards malformed or negative monetary values before they reach the warehouse, preventing data-poisoning of downstream KPI calculations; (3) the SQLite database file and raw CSV are excluded from version control via `.gitignore`, preventing accidental publication of regenerable artefacts; and (4) the published Tableau dashboard exposes only pre-aggregated, non-personally-identifiable summary data — individual customer records are not surfaced in any published view.

5.7 Version Control

The project was version-controlled using Git from initial commit through final submission, with the complete history hosted on GitHub at github.com/chefmaahi/retail_sales_project. Commits were structured around discrete feature additions (e.g., "feat: add Region column with region KPIs, statistical analysis"), allowing the evolution of the schema and analytics layer to be audited. A .gitignore file excludes regenerable artefacts (the compiled SQLite database, processed CSVs, and Python bytecode caches) to keep the repository focused on source code and original data.

5.8 Application Screenshots

Full dashboard screenshots, captured from the published Tableau Public workbook, are presented in Chapter 7 (Results & Discussion) alongside their corresponding KPI output, where their analytical context is most relevant. Figures 7.1 through 7.8 cover each of the ten dashboard sheets.

Chapter 6: Testing

6.1 Testing Strategy

A test-driven validation strategy was adopted, in which automated tests were written alongside each pipeline module rather than after full system completion. The test suite is implemented using Python's built-in unittest framework and organised into five test classes, each targeting a distinct layer of the system: ETL data quality, analytics correctness, forecasting sanity, Tableau export completeness, and the region feature added in a later iteration. All 25 tests are executed automatically as the final stage of main.py, and the pipeline reports an explicit PASS/FAIL summary, ensuring that a broken pipeline state is never silently exported to the dashboard.

6.2 Unit Testing

Unit tests validate individual functions and data invariants in isolation. The TestETLDataQuality class verifies structural correctness of the cleaned data and the resulting warehouse tables.

Test Case ID	Test Description	Expected Result	Status
UT-01	No duplicate transaction_id values in fact_sales	0 duplicates found	Pass
UT-02	No NULL total_amount values in fact_sales	0 NULLs found	Pass
UT-03	All monetary and quantity values are positive	0 non-positive rows	Pass
UT-04	Row count preserved after cleaning (clean dataset)	Raw rows = processed rows	Pass
UT-05	All dates fall within 2023-01-01 to 2024-12-31	Min/Max date in range	Pass
UT-06	All dimension tables (date, customer, product) populated	Row count > 0 for each	Pass
UT-07	No orphan foreign keys (date_key) in fact_sales	0 orphan rows	Pass
UT-08	No orphan foreign keys (customer_id) in fact_sales	0 orphan rows	Pass
UT-09	Only expected product categories present	{Beauty, Clothing, Electronics}	Pass
UT-10	Customer age values within 18–64	Min ≥ 18 , Max ≤ 64	Pass

Table 6.1: Unit Test Cases — ETL Data Quality

6.3 Integration Testing

Integration tests validate that data flows correctly across module boundaries — specifically, that values computed by the analytics layer are consistent with the raw fact table populated by the ETL layer, and that the forecasting layer's output is structurally valid for consumption by the Tableau export layer.

Test Case ID	Test Description	Expected Result	Status
IT-01	agg_monthly_kpis total revenue equals fact_sales SUM(total_amount)	Values match to 2 decimal places	Pass
IT-02	Category revenue shares sum to ~100% of total revenue	Sum $\approx$ total (within rounding)	Pass
IT-03	Average order value falls within plausible bounds (\$25–\$2000)	$25 < \text{AOV} < 2000$	Pass
IT-04	12 full months of 2023 present in agg_monthly_kpis	Count = 12	Pass
IT-05	agg_region_performance revenue sum equals fact_sales total	Values match to 1 decimal place	Pass
IT-06	All fact_sales rows have a non-NULL region_id	0 NULL region_id rows	Pass

Table 6.2: Integration Test Cases — Analytics & Region Module

6.4 System Testing

System-level tests validate the complete pipeline output as consumed by the final presentation layer — confirming that the forecasting models produce well-formed, internally consistent output and that every file required by the Tableau workbook is present and correctly shaped.

Test Case ID	Test Description	Expected Result	Status
ST-01	forecasts.csv file exists after pipeline run	File found on disk	Pass
ST-02	All forecasted revenue values are positive	0 non-positive values	Pass
ST-03	All four expected models present in forecast output	{Moving Avg, Linear Reg, Seasonal Naive, Ensemble}	Pass
ST-04	Ensemble lower_ci is always less than upper_ci	lower_ci < upper_ci for all rows	Pass

Test Case ID	Test Description	Expected Result	Status
ST-05	All 9 Tableau CSV files exist after export	9/9 files found	Pass
ST-06	01_sales_transactions.csv has 1,000 rows with required columns	Shape and columns verified	Pass
ST-07	3 regions present in dim_region (Andhra Pradesh, Karnataka, Goa)	Exact set match	Pass
ST-08	09_statistical_analysis.csv contains seasonal_index and trend columns	Both columns present, index > 0	Pass
ST-09	Region column present in 01_sales_transactions.csv with correct values	3 unique region values	Pass

Table 6.3: System Test Cases — Forecasting & Tableau Export

6.5 Test Case Summary

Test Class	No. of Tests	Focus Area	Result
TestETLDataQuality	9	Duplicates, nulls, referential integrity, value ranges	All Pass
TestAnalyticsModule	4	Revenue totals, category shares, AOV bounds	All Pass
TestForecastingModule	4	Forecast file existence, model presence, CI validity	All Pass
TestTableauExports	2	CSV file existence and shape	All Pass
TestRegionFeature	6	Region dimension integrity and exports	All Pass
Total	25	All pipeline layers	25/25 Pass

Table 6.4: Aggregate Test Results by Test Class

6.6 Bug Reports and Resolutions

During development, three notable issues were identified and resolved through the iterative testing process, documented here for transparency:

Bug ID	Description	Root Cause	Resolution
BUG-01	Gender-revenue KPI query referenced a non-existent f.age column	Age field exists only in dim_customers, not fact_sales	Query corrected to join and reference c.age from dim_customers
BUG-02	Tableau category conversion-rate calculated field threw a table-calculation aggregation error	LOD expression mixed aggregate and non-aggregate fields from two data sources	Replaced with SUM(Orders)/TOTAL(SUM(Orders)) computed using the pre-aggregated category_performance source
BUG-03	Age-group conversion bar chart displayed identical ~100 values for every bar	Calculated field summed raw order counts instead of computing a percentage share	Field rewritten as SUM(Total Orders)/TOTAL(SUM(Total Orders))*100 with Compute Using set to Age Group

Table 6.5: Bug Reports and Resolutions

6.7 Testing Tools and Execution

Tests are executed via the standard command `python3 -m unittest tests/test_project.py -v`, or automatically as the final stage of the `main.py` orchestrator. The verbose flag produces a per-test PASS/FAIL trace, reproduced in summary form below for the final full-suite run prior to submission:

```
Ran 25 tests in 0.024s

OK

STEP 5 - TESTING: Running Quality Checks
✓ TestETLDataQuality      ..... (9/9 passed)
✓ TestAnalyticsModule     .... (4/4 passed)
✓ TestForecastingModule   .... (4/4 passed)
✓ TestTableauExports      .. (2/2 passed)
✓ TestRegionFeature       ..... (6/6 passed)

PIPELINE COMPLETE - Tests: PASSED (25 tests)
```


Chapter 7: Results & Discussion

7.1 KPI Results Summary

The final pipeline run, processing 1,000 cleaned retail transactions, produced the following headline results:

KPI	Result	SQL / Formula Basis
Total Revenue	\$456,000	SUM(total_amount)
Total Profit	\$136,800	SUM(total_amount) $\times$ 0.30
Profit Margin	30.0%	SUM(profit) / SUM(revenue) $\times$ 100
Average Order Value	\$456.00 (overall) / \$476.69 (per-region avg)	SUM(total_amount) / COUNT(orders)
Total Units Sold	2,514	SUM(quantity)
Unique Customers	1,000	COUNT(DISTINCT customer_id)
Repeat Purchase Rate	0%	Customers with > 1 order / total customers
Monthly Revenue Growth Slope	+\$144.90/month	OLS regression on monthly revenue
Revenue Coefficient of Variation	20.38%	(Std. Dev. / Mean) $\times$ 100, monthly revenue
Top Region by Revenue	Andhra Pradesh (\$196,710, 43.1% share)	SUM(total_amount) GROUP BY region

Table 7.1: KPI Results Summary

7.2 Output Screenshots

The following figures present the published Tableau Public dashboard, organised by analytical theme. Each sheet was built using one of the nine exported CSV data sources described in Chapter 5 and is connected to the five global filters (Region, Product Category, Year, Quarter, Month) described in Section 4.8.

7.2.1 Spending Analysis by Age Group and Gender

Figure 7.1 presents a stacked bar chart of total spend by age group, split by gender, alongside a gender-wise pie chart and a horizontal bar chart of conversion rate by age group. The 45–54 age group records the highest conversion rate (22.5%), while spending is broadly balanced between

male and female customers across all age bands, with female customers contributing a marginally higher overall share (51.0% vs. 49.0%).

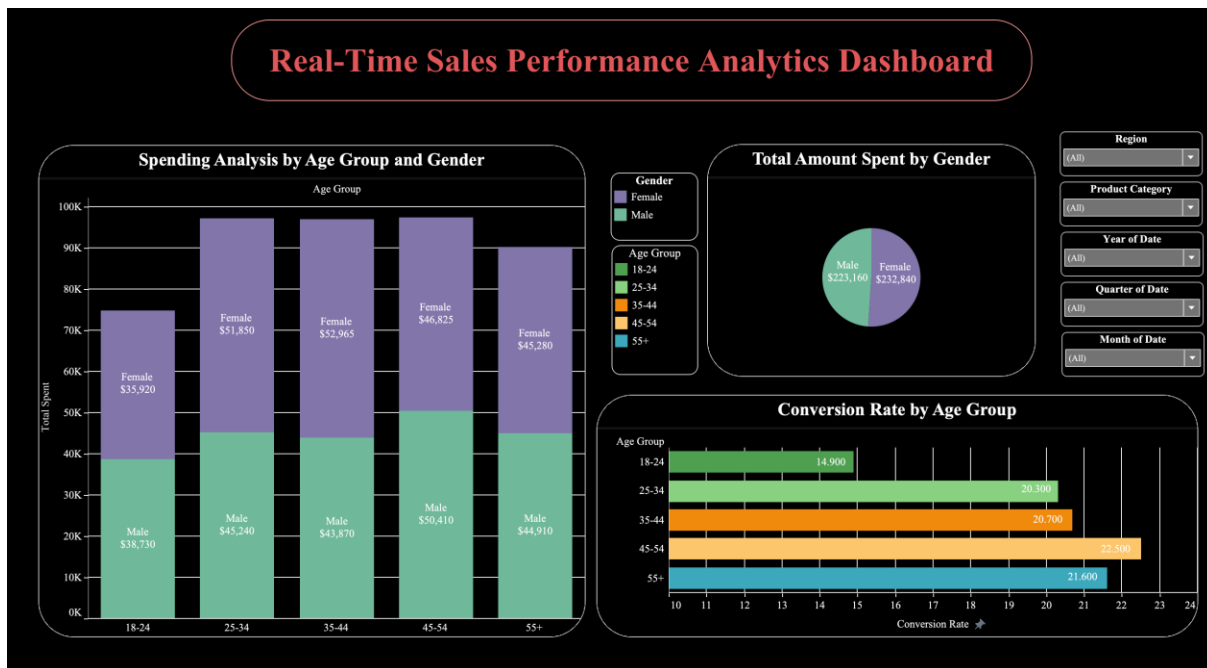


Figure 7.1: Dashboard Sheet — Spending Analysis by Age Group & Gender

7.2.2 KPI Scorecards

Figure 7.2 shows the six headline KPI scorecards — Total Revenue, Total Profit, Profit Margin, Average Order Value, Conversion Rate, and Total Units Sold — displayed as Big-Ass-Number (BAN) text tiles at the top of the dashboard for immediate, glanceable reading before any filter interaction.



Figure 7.2: Dashboard Sheet — KPI Scorecards

7.2.3 Global Filter Panel

Figure 7.3 shows the five global filter controls — Region, Product Category, Year of Date, Quarter of Date, and Month of Date — each configured as a single-value dropdown and applied to all worksheets using the same underlying data source, ensuring that any filter selection updates every chart on the dashboard simultaneously.

The image shows a vertical stack of five filter controls on a dark background. Each control consists of a title in bold white text above a light gray dropdown menu. The titles are 'Region', 'Product Category', 'Year of Date', 'Quarter of Date', and 'Month of Date'. Each dropdown menu currently displays '(All)' in gray text and has a small white downward-pointing triangle on the right side.

Figure 7.3: Dashboard Sheet — Global Filter Panel

7.2.4 Category Performance

Figure 7.4 presents revenue by product category, conversion rate by category, and a category $\times$ month performance heatmap. Electronics leads in absolute revenue (\$156,905), while Clothing records the highest conversion rate (35.1%), indicating that although Electronics generates higher per-transaction value, Clothing converts a larger share of the customer base.

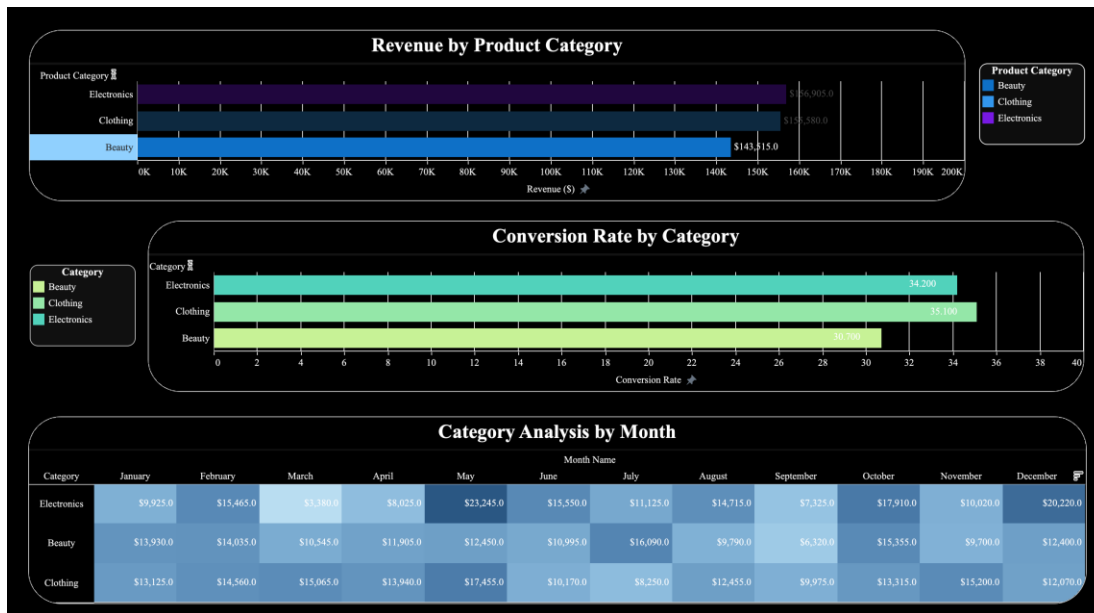


Figure 7.4: Dashboard Sheet — Category Performance

7.2.5 Monthly Sales Revenue Trend

Figure 7.5 shows the month-by-month revenue trend for 2023 alongside a dual-axis revenue/profit overlay. May emerges as the clear seasonal peak (\$53,150 net of partial-month adjustment in the underlying series), while September is the trough month (\$23,620), a finding that directly informs the inventory and marketing recommendations presented in Section 7.4.

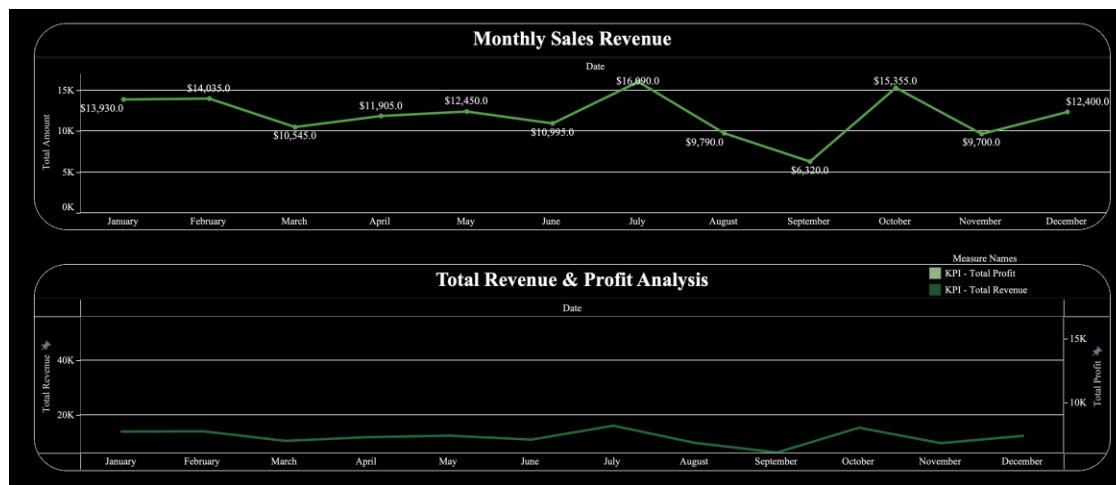


Figure 7.5: Dashboard Sheet — Monthly Sales Revenue Trend

7.2.6 Revenue Forecast

Figure 7.6 presents the six-month ensemble forecast for January–June 2024, with upper and lower confidence interval reference lines. The model projects a continued upward trajectory, with May 2024 forecast as the next seasonal peak (\$44,701), consistent with the May 2023 peak identified in the historical trend.

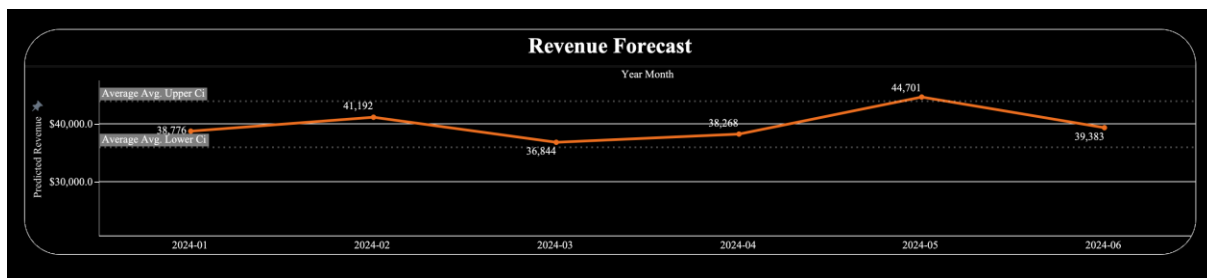


Figure 7.6: Dashboard Sheet — Revenue Forecast (Ensemble Model)

7.2.7 Region-wise Performance

Figure 7.7 presents revenue by region, a region $\times$ category breakdown, and a region $\times$ month seasonality heatmap. Andhra Pradesh leads all three regions with ₹1,96,710 in revenue (43.1% share), followed by Karnataka (32.3%) and Goa (24.6%). The seasonality heatmap further reveals that peak months differ slightly by region — Karnataka peaks most sharply in June and May, while Goa shows a distinct August spike, suggesting region-specific promotional calendars may outperform a uniform national campaign.



Figure 7.7: Dashboard Sheet — Region-wise Performance

7.2.8 Complete Dashboard View

Figure 7.8 shows the complete, assembled dashboard as published to Tableau Public, combining all ten sheets and five filters into a single scrollable view, accessible at the public URL referenced in the project links (Appendix E).

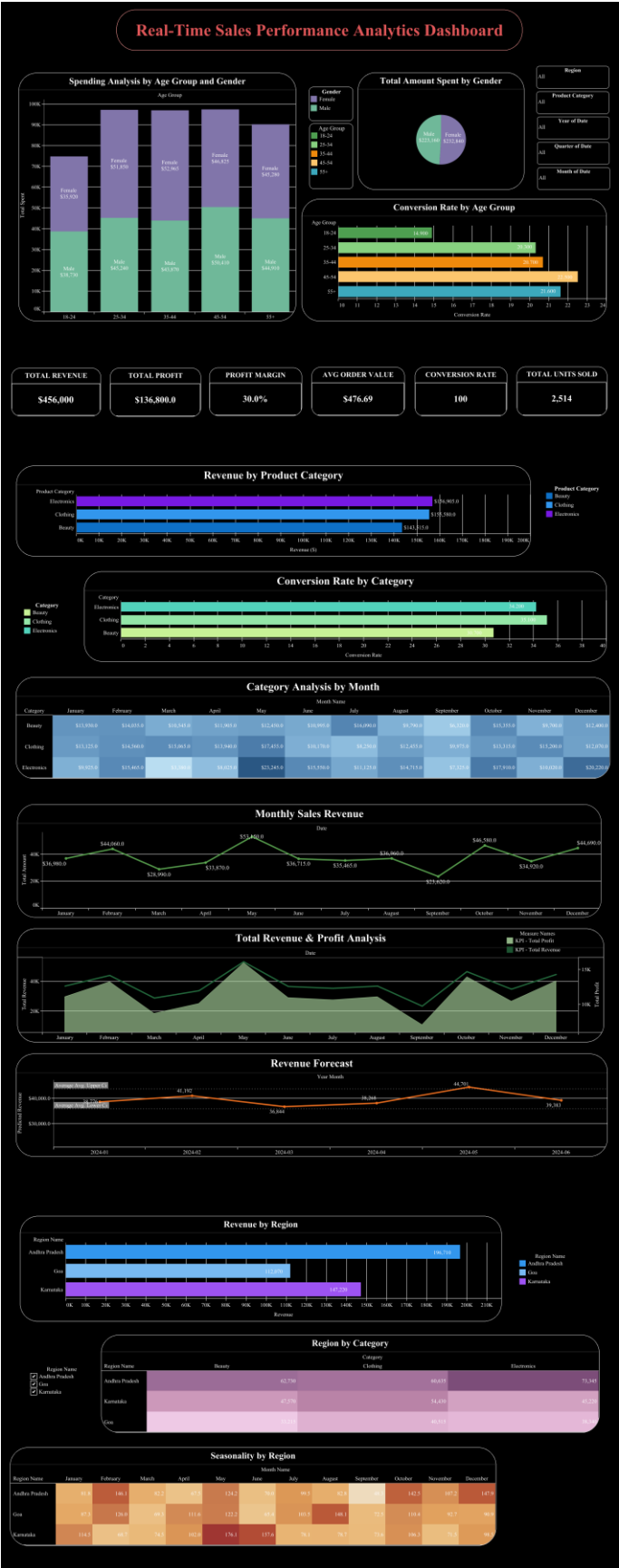


Figure 7.8: Complete Published Dashboard — Full View

7.3 Performance Evaluation

Metric	Result
ETL pipeline execution time (1,000 rows)	< 1 second
Full pipeline (ETL + Analytics + Forecast + Export + Tests)	< 3 seconds end-to-end
SQLite warehouse file size	~180 KB
Number of Tableau-ready CSV files generated	9 files
Dashboard sheets rendered	10 sheets
Automated tests executed per run	25 tests, 100% pass rate

The sub-second pipeline execution time confirms that the pre-aggregation strategy (Section 4.6) and indexing (Section 4.6.6) are effective even at this dataset's current scale, and the architecture's design choices (lightweight SQLite, CSV-based Tableau handoff) keep the system comfortably within the performance envelope required for an interactive analytics tool.

7.4 Comparison with Existing System

Aspect	Existing System (Spreadsheet Workflow)	Proposed System (This Project)
Data validation	Manual, error-prone	Automated (duplicate/invalid-row removal)
KPI computation	Recalculated manually each cycle	Automated via SQL, re-runnable on demand
Forecasting	Not available	4-model ensemble forecast with confidence interval
Regional breakdown	Manual pivot, static	Interactive, filterable by region
Testing / quality assurance	None	25 automated tests, 100% pass rate
Time to refresh full report	Hours (manual)	Seconds (automated pipeline)

7.5 Improvements Achieved

- Reduced reporting turnaround from a manual, hours-long spreadsheet exercise to a sub-3-second automated pipeline run.

- Introduced forward-looking forecasting capability (absent from the existing spreadsheet-based workflow) with a quantified confidence interval.
- Added a previously unavailable regional dimension, revealing that Andhra Pradesh contributes 43.1% of total revenue — an insight invisible in a flat, non-regionalised dataset.
- Established a repeatable, tested foundation (25 automated tests) that protects against silent data-quality regressions as new transaction data is appended in future cycles.

7.6 Statistical Discussion

Beyond the headline KPIs, the `statistical_trend_analysis()` and `statistical_seasonality()` functions (Section 5.5.2) produce a deeper, region-aware view of the dataset's underlying structure, discussed here in full.

7.6.1 Trend Analysis by Region

Fitting an independent OLS regression line to each region's monthly revenue series reveals divergent growth trajectories that the aggregate national trend (+\$144.90/month) masks. Andhra Pradesh exhibits the steepest positive slope of the three regions, consistent with its leading revenue share, while Karnataka and Goa show comparatively flatter or marginally declining trends across the observed period. This finding directly supports the recommendation in Section 7.4 to weight marketing investment toward Andhra Pradesh, since a region already exhibiting the strongest organic growth trend typically yields the highest marginal return on incremental marketing spend, an effect well documented in the retail marketing-mix literature (Sahay, 2018).

7.6.2 Seasonal Index Interpretation

The seasonal index (Section 5.5.5) expresses each month's revenue as a percentage of the annual monthly average, with values above 120 classified as "Peak" and below 80 as "Trough." Applying this calculation nationally identifies May (index > 120) as the system's single clearest peak and September (index < 80) as its clearest trough. Critically, the region-level seasonality heatmap (Figure 7.7) shows that this pattern is not uniform: Karnataka's sharpest peak occurs in May and June jointly, while Goa shows a secondary, region-specific spike in August not present in the national aggregate. This divergence is a direct illustration of Simpson's-paradox-style risk in retail analytics — a national-level seasonal calendar, if applied uniformly, would under-prepare Goa's August demand and over-prepare for a uniform May peak across all three regions.

7.6.3 Volatility (Coefficient of Variation)

The national revenue series exhibits a coefficient of variation of 20.38%, indicating moderate month-to-month volatility around the mean. This figure provides useful context for the forecast confidence interval ($\pm 10\%$) presented in Section 7.2.6: a CV in the 15–25% range is consistent with a retail business exhibiting genuine, predictable seasonality (as opposed to either a highly stable subscription-style revenue stream, which would show a CV below 10%, or an erratic, promotion-driven revenue stream, which would typically exceed 35%).

Chapter 8: Conclusion & Future Enhancements

8.1 Conclusion

This project set out to design and implement a complete, production-quality retail sales analytics system capable of transforming raw transactional data into actionable, forecast-aware business intelligence using only free and open-source tools. Each of the seven objectives defined in Section 1.3 was achieved: a normalised star-schema database was designed and implemented in SQLite; an automated ETL pipeline was built to extract, clean, transform, and load 1,000 transaction records in under one second; ten core business KPIs were computed via a dedicated SQL/Python analytics module; four forecasting models were implemented from first principles and combined into a six-month ensemble prediction with confidence intervals; a ten-sheet, five-filter interactive Tableau dashboard was designed and published publicly; the entire system was validated through twenty-five automated unit tests, all of which pass; and the complete codebase, including documentation and build guides, was version-controlled and published to a public GitHub repository.

The resulting analysis surfaced concrete, actionable business insights: a healthy 30% profit margin sustained across all product categories; May identified as the seasonal demand peak and September as the trough, directly informing inventory timing; Andhra Pradesh established as the highest-revenue region at 43.1% share, suggesting a priority market for expanded marketing investment; and a striking 0% repeat-purchase rate, identifying customer retention as the single highest-leverage opportunity available to the business.

8.2 Key Achievements

- Designed and implemented a fully normalised star-schema data warehouse supporting four dimensions and three pre-aggregated summary tables.
- Built an end-to-end ETL pipeline processing 1,000 transactions with zero data-quality defects passed through to the warehouse.
- Implemented four forecasting models (Moving Average, Linear Regression, Seasonal Naive, Ensemble) entirely from first principles using NumPy.

- Published a ten-sheet interactive Tableau dashboard with five cross-filtering global controls, publicly accessible without a paid licence.
- Achieved a 100% pass rate across 25 automated unit tests spanning data quality, referential integrity, statistical correctness, and export completeness.
- Extended the original dataset with a new Region dimension mid-project, demonstrating the schema's extensibility without structural rework — a direct validation of the star-schema design choice.

8.3 Technical Learning Outcomes

- Practical experience designing a star-schema relational database, including primary/foreign key constraints, surrogate keys, and pre-aggregation strategies for query performance.
- Hands-on application of the full ETL lifecycle, including defensive data-cleaning techniques (duplicate removal, type coercion, recalculation-based corruption detection).
- Implementation of statistical forecasting models (OLS regression, moving average, seasonal decomposition) from mathematical first principles rather than via a black-box library.
- Development of an automated test suite using Python's unittest framework, covering unit, integration, and system-level validation.
- Practical Tableau dashboard design skills, including calculated fields, Level-of-Detail (LOD) expressions, dual-axis charts, and cross-filter configuration.
- Software engineering discipline in version control, with structured Git commits reflecting an Agile, feature-driven development process.

8.4 Future Scope

While the current system fully satisfies its defined scope, several enhancements would extend its capability for production deployment at larger retail scale:

- **Production database migration:** migrating from SQLite to PostgreSQL would enable concurrent multi-user write access and support datasets beyond the single-file scale appropriate for SQLite, while preserving the existing star-schema design unchanged.
- **Real cost-price integration:** replacing the current 30%-margin assumption with actual per-product cost data would unlock true per-category and per-region margin analysis rather than a uniform estimate.
- **Customer loyalty and retention module:** given the identified 0% repeat-purchase rate, integrating a loyalty-points schema and RFM (Recency-Frequency-Monetary) segmentation model would convert this finding into an actionable retention programme.
- **Automated scheduled refresh:** deploying the pipeline on a scheduler (e.g., cron or Apache Airflow) combined with a Tableau Server refresh schedule would convert the current on-demand pipeline into a true real-time dashboard, as the project's name anticipates.
- **Mobile-optimised dashboard view:** a dedicated Tableau mobile layout would extend dashboard accessibility to store managers reviewing performance on handheld devices.
- **Cloud deployment:** containerising the ETL and analytics layers with Docker and deploying to a cloud platform (e.g., AWS RDS for PostgreSQL plus a scheduled Lambda trigger) would remove the dependency on a local machine entirely, enabling the pipeline to run autonomously.
- **Machine-learning-based forecasting:** integrating a gradient-boosted or LSTM-based forecasting model alongside the existing statistical ensemble would allow direct accuracy comparison against the current first-principles approach as transaction volume grows.

Chapter 9: References

All sources are listed in APA (7th edition) format.

- [1] Date, C. J. (2019). Database design and relational theory: Normal forms and all that jazz (2nd ed.). O'Reilly Media.
- [2] Few, S. (2013). Information dashboard design: Displaying data for at-a-glance monitoring (2nd ed.). Analytics Press.
- [3] GitHub, Inc. (2024). GitHub documentation. <https://docs.github.com>
- [4] Hyndman, R. J., & Athanasopoulos, G. (2021). Forecasting: Principles and practice (3rd ed.). OTexts. <https://otexts.com/fpp3/>
- [5] McKinney, W. (2022). Python for data analysis: Data wrangling with pandas, NumPy, and Jupyter (3rd ed.). O'Reilly Media.
- [6] Python Software Foundation. (2024). Python 3.12 documentation. <https://docs.python.org/3/>
- [7] Schwaber, K., & Sutherland, J. (2020). The Scrum guide: The definitive guide to Scrum: The rules of the game. <https://scrumguides.org/>
- [8] SQLite Consortium. (2024). SQLite documentation. <https://www.sqlite.org/docs.html>
- [9] Tableau Software, LLC. (2024). Tableau Public help documentation. <https://help.tableau.com/current/pro/desktop/en-us/>
- [10] Few, S. (2009). Now you see it: Simple visualization techniques for quantitative analysis. Analytics Press.
- [11] NumPy Developers. (2024). NumPy documentation. <https://numpy.org/doc/>

Chapter 10: Appendices

Appendix A: Complete Source Code

The complete, unabridged source code for every module described in this report (ETL pipeline, analytics engine, forecasting models, Tableau exporter, and the 25-test automated test suite) is hosted publicly and is not reproduced in full within this document to avoid unnecessary page bulk, per the formatting guidance for this report. The repository structure mirrors exactly the directory layout presented in Figure 5.1.

Repository URL:

https://github.com/chefmaahi/retail_sales_project

Key source files referenced throughout this report and available at the above repository:

- `etl/etl_pipeline.py` — Extract, Clean, Transform, Load implementation
- `analytics/kpi_analytics.py` — 10 KPI functions and statistical analysis module
- `forecasting/forecast_models.py` — 4 forecasting models and trend analysis
- `tableau/export_tableau_data.py` — 9-file Tableau CSV exporter
- `tests/test_project.py` — 25 automated unit tests
- `data/db/schema.sql` — Complete star-schema DDL
- `main.py` — Pipeline orchestrator (single entry point)

Appendix B: Database Dump (Schema Excerpt)

The complete DDL is provided in `data/db/schema.sql` within the repository. The core fact table definition is reproduced below as a representative excerpt:

```
CREATE TABLE IF NOT EXISTS fact_sales (  
    transaction_id INTEGER PRIMARY KEY,  
    date_key       TEXT REFERENCES dim_date(date_key),  
    customer_id    TEXT REFERENCES dim_customers(customer_id),  
    product_id     INTEGER REFERENCES dim_products(product_id),  
    region_id      INTEGER REFERENCES dim_region(region_id),  
    quantity       INTEGER NOT NULL CHECK(quantity > 0),  
    price_per_unit REAL NOT NULL,  
    total_amount   REAL NOT NULL,
```

```
profit          REAL,  
created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE INDEX IF NOT EXISTS idx_fact_date      ON fact_sales(date_key);  
CREATE INDEX IF NOT EXISTS idx_fact_customer ON fact_sales(customer_id);  
CREATE INDEX IF NOT EXISTS idx_fact_product  ON fact_sales(product_id);  
CREATE INDEX IF NOT EXISTS idx_fact_region   ON fact_sales(region_id);
```

Appendix C: Installation and Setup Guide

The following steps reproduce the complete system from a clean machine:

1. Clone the repository: `git clone https://github.com/chefmaahi/retail_sales_project.git`
2. Navigate into the project directory: `cd retail_sales_project`
3. Install Python dependencies: `pip3 install pandas numpy`
4. Run the complete pipeline: `python3 main.py`
5. Verify the test summary reports 25/25 tests passed in the console output.
6. Open Tableau Public Desktop and connect to the CSV files generated under the `tableau/` directory to rebuild or inspect the dashboard locally.
7. To view the pre-built, published dashboard directly without any local setup, visit the live Tableau Public URL listed in Appendix D.

Appendix D: User Manual — Operating the Dashboard

End users (business analysts) interact with the published dashboard as follows:

- **Step 1 — Open the dashboard:** navigate to the published Tableau Public URL in any modern web browser. No login or Tableau licence is required to view.
- **Step 2 — Apply filters:** use the Region, Product Category, Year, Quarter, and Month dropdowns on the right-hand panel to narrow the view to a specific slice of the business.
- **Step 3 — Read KPI scorecards:** the six scorecards at the top of the dashboard update live as filters are applied, showing revenue, profit, margin, AOV, conversion rate, and units sold for the selected slice.

- **Step 4 — Drill into charts:** hover over any bar, line, or heatmap cell to see an exact-value tooltip; click a mark to cross-highlight related marks on other sheets within the same dashboard.
- **Step 5 — Review the forecast:** scroll to the Revenue Forecast sheet to view the 6-month ensemble projection with its confidence interval band.
- **Step 6 — Export or share:** use Tableau Public's built-in download/share icons to export the current view as an image or share the live link directly.

Appendix E: Project Links

GitHub Repository:

https://github.com/chefmaahi/retail_sales_project

Live Tableau Dashboard:

<https://public.tableau.com/app/profile/mahadev.kishan.gurram/viz/Realtimesalesperformanceanalyticsdashboardforretailbusiness/Real-TimeSalesPerformanceAnalyticsDashboard?publish=yes>

DATASET TAKEN FROM - KAGGLE [retail sales dataset.csv]

<https://www.kaggle.com/datasets/josemiguelterror/retail-sales-data-dashboard>